

Computer vision e GraphRAG dal riconoscimento visivo al knowledge graph

Università degli Studi di Verona

Alberto Dosso

VR485765

22 settembre 2026

Indice

1	Introduzione	1
2	Dettagli tecnici	2
2.1	Rete Neurale	2
2.1.1	Layers	3
2.1.2	Pesi	3
2.1.3	Bias	3
2.2	Dataset	4
2.2.1	Qualità di un Dataset	4
2.3	Training	5
2.3.1	Training from scratch	5
2.3.2	Transfer Learning	5
2.3.3	Fine-tuning	6
2.4	Epoca	7
2.5	Pytorch	8
2.5.1	Tensori	8
2.5.2	Grafici	8
3	YOLO	9
3.1	YOLO26	10
3.1.1	Versioni	10
3.1.2	Metriche di valutazione	10
3.1.3	Determinismo delle metriche	14
3.2	Script	15
3.2.1	Dataset	15
3.2.2	Training	15
3.2.3	Risultati	16
3.2.4	Inferenza	19

4	SAM3	20
4.1	Generazione del Dataset	21
4.2	Script	21
4.2.1	Librerie	22
4.2.2	Configurazione	22
4.2.3	Main	23
4.2.4	Risultati	24
4.2.5	Ottimizzazioni	25
4.3	Conclusioni	25
5	Segformer	26
6	VLM	28
6.1	Introduzione	28
6.2	Qwen 3	29
6.2.1	Fine-tuning	29
6.3	Gemma 4	32
6.3.1	Risultati	32
7	Grounding DINO	33
8	Test	34
8.1	Ground truth	34
8.1.1	Abbinamento tra predizioni e ground truth	35
8.1.2	Confusion matrix	37
8.1.3	Metriche	38
8.2	Confronto tra modelli	39
8.2.1	Dataset di addestramento	39
8.2.2	Annotazione con SAM3	40
8.2.3	YOLO detection	41
8.2.4	YOLO segmentation	42
8.2.5	SegFormer	43
8.2.6	GroundingDINO	44
8.2.7	SAM3	45
8.3	Dai modelli al sistema	46

9	Graph RAG	47
9.1	Ciclo di vita	47
9.2	Database vettoriali	49
9.3	Database a Grafo	49
9.4	Integrazione di SAM3	50
9.5	Pipeline	51
9.5.1	Architettura	51
9.5.2	Segmentazione	52
9.5.3	Filtri di qualità	53
9.5.4	Riconoscimento a catalogo	55
9.5.5	Modello dei dati	57
9.5.6	Costruzione	58
9.5.7	Rappresentazione grafica	59
9.5.8	Limiti noti	60
	Bibliografia	61

1. Introduzione

L'obiettivo di questa ricerca è quello di analizzare e migliorare il rilevamento di capi d'abbigliamento all'interno di un'immagine utilizzando modelli di Machine Learning. In questi ultimi anni l'avanzamento delle tecnologie riguardanti il campo dell'Intelligenza Artificiale ha avuto una crescita esponenziale, migliorando la comprensione che loro hanno del mondo, e la risoluzione di problemi complessi soprattutto grazie all'incremento della mole di dati.

La proposta è quella di usare modelli nel campo della visione artificiale, uniti ad algoritmi di Deep Learning, per migliorare il più possibile il riconoscimento di diverse le categorie di capi d'abbigliamento.

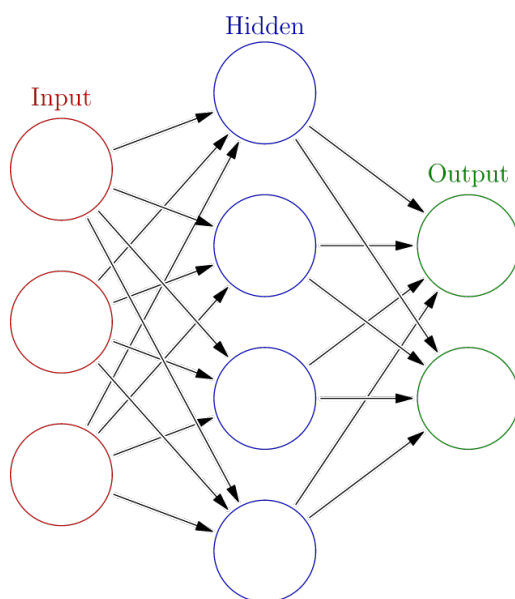
2. Dettagli tecnici

2.1 Rete Neurale

Una Rete Neurale è un modello matematico che simula il funzionamento della mente biologica umana, ed è utilizzata per tentare di risolvere problemi ingegneristici di intelligenza artificiale. [1]

Nel nostro caso, per la risoluzione di problemi di visione artificiale, utilizzeremo le reti neurali multi livello (o multi layer), chiamate anche MLP (Multi Layer Perceptron), che appartengono ad una sotto categoria del Machine Learning chiamata Deep Learning.

Da notare che non tutti i modelli di Intelligenza Artificiale sono basati sulle reti neurali; esistono anche altre tecnologie utilizzate nel mondo del Machine Learning come i Decision Trees, i quali creano diagrammi di flusso basati su condizioni specifiche, oppure le Random Forest, cioè l'unione di più Decision Tree indipendenti.



[2]

Figura 2.1: Percettrone Multistrato

2.1.1 Layers

In una Rete Neurale i layers rappresentano i diversi strati della rete. In ogni strato sono presenti uno o più neuroni chiamati unità.

Sono generalmente presenti tre strati :

1. Input : con unità che rappresentano i campi di input.
2. Hidden Layers : con uno o più strati.
3. Output : con una o più unità che rappresentano i campi obiettivo.

I dati di input vengono presentati nel primo strato e i valori vengono propagati da ciascun neurone a ogni neurone nello strato successivo, ed alla fine viene fornito un risultato nello strato di output. [3]

2.1.2 Pesì

Le unità sono connesse con varie intensità di connessione, dette pesi.

Tutti i pesi inizialmente sono casuali e le risposte ottenute dalla rete risulteranno probabilmente prive di senso.

La rete apprende generando una previsione per ogni input dato e correggendo i pesi ogni volta che viene sbagliata una previsione. Questo processo, anche detto *epoca*, viene ripetuto molte volte e la rete continua a migliorare le sue previsioni finché uno o più criteri di interruzione non vengono soddisfatti. [3]

2.1.3 Bias

Il parametro di bias è quello che, durante il calcolo matematico che avviene all'interno della singola unità (neurone) permette di aggiungere un'errore al modello, considerato troppo semplice, con l'obiettivo di catturare una maggiore complessità dei dati.

2.2 Dataset

Un Dataset è una raccolta strutturata di dati, che in questo caso servirà per la fase di addestramento (training) dei nostri modelli.

In generale, un Dataset per l'addestramento di Reti Neurali è suddiviso in :

- Training set : immagini usate per il training del nostro modello.
- Validation set : immagini utilizzate durante l'addestramento, per valutare le prestazioni del modello su dati non visti e per effettuare la messa a punto dei parametri.
- Test set : un insieme di immagini utilizzate alla fine dell'addestramento, per valutare le prestazioni finali del modello, su dati completamente nuovi che non sono stati utilizzati per il training.

Il Validation set è opzionale, ma molto utile per migliorare la qualità del training del modello.

2.2.1 Qualità di un Dataset

La qualità di un Dataset è data dalla correttezza delle sue annotazioni, cioè dalla coerenza di quello che mi stanno dicendo i dati al suo interno, ma anche dalla varietà dei dati al suo interno, perché avrò molti più esempi da mostrare al mio modello nella fase di addestramento.

Perciò se i dati sono coerenti ai problemi del mondo reale di cui si occuperà il modello, impararne i pattern e le correlazioni consentirà al modello addestrato di fare previsioni accurate su nuovi dati.

2.3 Training

L'addestramento di un modello è la fase del Machine Learning (ML) in cui avviene l'apprendimento. Nel ML, l'apprendimento implica la regolazione dei parametri di un modello ML. Questi parametri includono *pesi* e *bias* nelle funzioni matematiche che costituiscono i loro algoritmi.

Durante questa fase, ogni volta che un input arriva ad un nodo, viene moltiplicato per il peso (weight) associato a quella specifica connessione, e la somma di tutti questi input con un certo valore di bias ne determinerà poi i valori in output.

Dal punto di vista matematico, l'obiettivo di questo apprendimento è quello di ridurre al minimo una funzione di perdita che quantifica l'errore degli output nelle attività di addestramento. Quando l'output della funzione di perdita scende al di sotto di una soglia predeterminata, ovvero l'errore del modello nelle attività di addestramento è sufficientemente piccolo, il modello viene considerato "addestrato". [4]

2.3.1 Training from scratch

In questa tipologia di training, il modello viene addestrato su dati annotati senza partire da pesi pre-trainati, quindi senza usare come base un modello già allenato. Partendo da zero, ovvero from scratch, i pesi non sono definiti ma vengono assegnati dei numeri casuali che, soltanto durante il processo di training, verranno modificati per ottimizzare le prestazioni.

Questo tipo di training è valido quando abbiamo a disposizione un numero considerevole di dati annotati e l'utilizzo del modello è molto specifico.

2.3.2 Transfer Learning

In scenari caratterizzati da dataset limitati o per ottimizzare tempi e risorse computazionali, è prassi comune ricorrere al transfer learning, inizializzando la rete con pesi pre-addestrati.

Questo approccio si fonda sulla natura gerarchica dell'apprendimento profondo: i primi layer della rete estraggono caratteristiche universali delle immagini, come bordi, variazioni di colore, e texture, mentre solo gli strati più profondi si specializzano su task specifici. Sfruttando la rappresentazione generale appresa precedentemente su dataset massivi, si evita di dover ricostruire l'estrattore di feature da zero, concentrando il nuovo addestramento solo su ciò che è strettamente legato al nuovo dominio.

I vantaggi rispetto a un addestramento from scratch sono notevoli: una drastica riduzione dei dati annotati necessari, una convergenza più rapida, con conseguente abbattimento dei costi computazionali, ed infine migliore capacità di generalizzazione. Il limite principale è invece legato al cambio di dominio, dove il trasferimento di conoscenza perde di efficacia se il nuovo dominio si discosta eccessivamente da quello di partenza.

2.3.3 Fine-tuning

Mentre il transfer learning rappresenta il paradigma concettuale, il fine-tuning ne costituisce l'implementazione pratica, in cui l'addestramento non parte da pesi inizializzati casualmente, ma da uno stato pre-ottimizzato che viene gradualmente adattato al nuovo dominio di riferimento. Operativamente, si distinguono due approcci: nel fine-tuning *completo* l'intera architettura viene aggiornata, garantendo massima flessibilità a fronte di un elevato onere computazionale; nel fine-tuning *parziale*, invece, i layer iniziali vengono "congelati" per preservare l'estrazione di feature generali, limitando l'aggiornamento agli strati terminali, spesso sostituiti per adattarsi alle nuove classi.

Un aspetto critico in questo processo è la calibrazione del learning rate, che deve essere sensibilmente inferiore rispetto al training from scratch. Aggiornamenti troppo aggressivi distruggerebbero infatti i delicati equilibri dei pesi originali, innescando il cosiddetto *catastrophic forgetting*, cioè la perdita irreversibile della conoscenza pregressa. Al contempo, occorre bilanciare attentamente la durata dell'addestramento per non incappare nell'overfitting, specialmente se il nuovo dataset è di dimensioni ridotte.

Infine, l'enorme scala raggiunta dai recenti modelli linguistici e multimodali ha reso spesso quasi impraticabili gli approcci tradizionali. Sono così emerse le tecniche di Parameter-Efficient Fine-Tuning (PEFT), che ottimizzano solo una frazione infinitesimale dei pesi mantenendo inalterata l'architettura originale. Una delle metodologie più efficaci in questo ambito è LoRA, approfondita nella Sezione 6.2.1.

2.4 Epoca

Un'epoca rappresenta un passaggio completo del dataset attraverso l'algoritmo in fase di learning, è un concetto fondamentale nel training delle reti neurali, che permette al modello di imparare continuando costantemente a rivedere le stesse immagini, o i medesimi elementi del dataset.

Il numero di epoch si specifica solitamente in ogni procedura di training e determina quante volte il modello imparerà dal set di train del dataset, influenzando le performance e la qualità del modello.

Scegliere il numero di epoch corretto è molto importante perché se viene scelto in maniera errata può creare due problemi:

- Underfitting: il modello non è stato trainato per un numero sufficiente di epochs. Le performances non sono adeguate né sul set di training né su quello di test.
- Overfitting: il modello è stato trainato per troppi epochs. In questo caso memorizza i dati di training in maniera troppo approfondita e perde le sue abilità di analizzare i nuovi dati. Ha una precisione eccellente sul training set ma scarsa sul dataset di validazione.

Una tecnica comune per evitare l'overfitting è quella dell'early stopping, nella quale il training viene fermato quando le performances sul set di validazione smettono di migliorare. A volte è possibile specificare un parametro chiamato patience, che specifica dopo quanti epochs "non migliorativi" il training deve fermarsi. [5].

2.5 Pytorch

PyTorch è un framework di deep learning open source basato su software usato per creare reti neurali. Combina la libreria di machine learning (ML) di Torch con un'API di alto livello basata su Python. La sua flessibilità e facilità d'uso, tra gli altri benefici, lo hanno reso il principale framework di ML per le comunità accademiche e di ricerca.

2.5.1 Tensori

In qualsiasi algoritmo di machine learning, anche quelli applicati a informazioni apparentemente non numeriche come suoni o immagini, i dati devono essere rappresentati numericamente. In PyTorch ciò viene ottenuto tramite tensori che fungono da unità fondamentali di dati utilizzate per il calcolo sulla piattaforma.

Dal punto di vista linguistico, “tensore” è un termine generico che si riferisce ad alcune entità matematiche più familiari come :

- Un vettore è un tensore monodimensionale contenente più scalari dello stesso tipo. Una tupla è un tensore monodimensionale contenente diversi tipi di dati.
- Una matrice è un tensore bidimensionale contenente più vettori dello stesso tipo.
- I tensori con tre o più dimensioni, come i tensori tridimensionali usati per rappresentare le immagini RGB negli algoritmi di computer vision, sono collettivamente denominati tensori N-dimensionali.

Oltre a codificare gli input e gli output di un modello, i tensori di PyTorch codificano i parametri del modello: pesi e gradienti che vengono ”appresi” nella machine learning.

2.5.2 Grafici

I grafici di calcolo dinamico (DCG) sono il modo in cui i modelli di deep learning vengono rappresentati in PyTorch. Nel contesto del deep learning, questi grafici traducono essenzialmente il codice di una rete neurale in un diagramma di flusso che indica le operazioni eseguite in ciascun nodo e le dipendenze tra diversi strati nella rete, cioè la disposizione di fasi e sequenze che trasformano i dati di input in dati di output. [6]

3. YOLO

YOLO, ovvero "You Only Look Once", è un diffuso algoritmo di rilevamento di oggetti in tempo reale.

YOLO unifica quello che un tempo era un processo a più fasi, utilizzando un'unica rete neurale per eseguire sia la classificazione sia la previsione dei riquadri di delimitazione (bounding box) degli oggetti rilevati. Ogni bounding box è definita dalle coordinate che ne identificano posizione e dimensioni all'interno dell'immagine, oltre a un punteggio di confidenza e all'etichetta della classe dell'oggetto. In questo modo il modello è in grado di localizzare e classificare simultaneamente tutti gli oggetti presenti in una singola inferenza, formulando il problema come una regressione diretta delle coordinate dei bounding box e delle probabilità di appartenenza alle diverse classi.

È fortemente ottimizzato per le prestazioni di rilevamento e può operare molto più velocemente rispetto all'esecuzione di due reti neurali distinte per rilevare e classificare gli oggetti separatamente. Ciò avviene riadattando i classificatori di immagini tradizionali per la specifica attività di regressione volta a individuare i bounding box degli oggetti. [7]

3.1 YOLO26

YOLO26 è l'ultima famiglia unificata di modelli di visione in tempo reale. Introduce l'inferenza nativa end-to-end, una head di rilevamento più leggera, una ricetta di addestramento aggiornata e head specifiche per compito per rilevamento, segmentazione, stima della posa, classificazione. [8]

3.1.1 Versioni

I modelli YOLO26, offrono diversi tagli per ogni versione del modello, passando dalla più piccola denominata "nano" (YOLO26n), alla più grande "extra large" (YOLO26x), nei quali differenza sta nella loro dimensione a livello di parametri. Questa variazione di parametri ci permette di aumentare le prestazioni, e quindi la qualità del modello.

3.1.2 Metriche di valutazione

Dopo aver addestrato un modello YOLO, dobbiamo misurarne le prestazioni ed eventualmente perfezionarlo per colmare le lacune. La valutazione utilizza metriche come precision e la recall, le quali unite alla mAP ci permettono di quantificare l'accuratezza del modello.

Model	mAP ₅₀	mAP ₉₅	Speed	Speed	params (M)
			CPU (ms)	GPU (ms)	
YOLO26n	40.9	40.1	38.9 ± 0.7	1.7 ± 0.0	2.4
YOLO26s	48.6	47.8	87.2 ± 0.9	2.5 ± 0.0	9.5
YOLO26m	53.1	52.5	220.0 ± 1.4	4.7 ± 0.1	20.4
YOLO26l	55.0	54.4	286.2 ± 2.0	6.2 ± 0.2	24.8
YOLO26x	57.5	56.9	525.8 ± 4.0	11.8 ± 0.2	55.7

[8]

Precision

La precision è la frazione di istanze rilevanti tra le istanze recuperate.

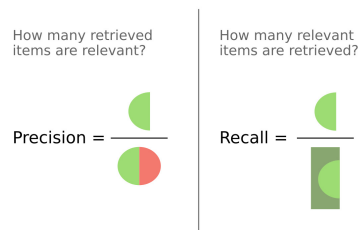
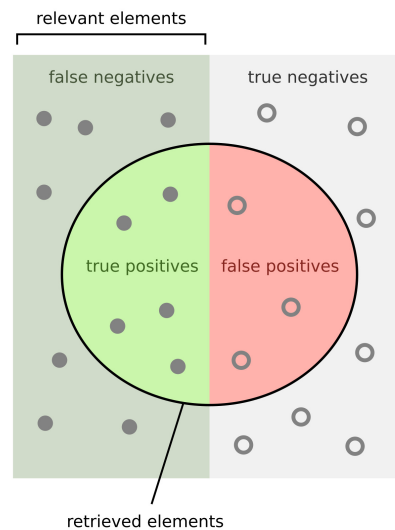
$$Precision = \frac{\text{Istanze rilevanti recuperate}}{\text{Tutte le istanze recuperate}} = \frac{TP}{TP + FP}$$

Recall

La recall (anche nota come sensibilità) è la frazione di istanze rilevanti che sono state recuperate.

$$Recall = \frac{\text{Istanze recuperate rilevanti}}{\text{Tutte le istanze rilevanti}} = \frac{TP}{TP + FN}$$

Sia la precision che la recall si basano sull'output generato dal nostro modello. Queste metriche possono essere meglio rappresentate tramite questa immagine.



[9]

F1 score

L'F1 score mette in rapporto tra la precision e la recall, offrendoci una migliore comprensione del bilanciamento del nostro modello.

mAP

La precisione media (mean average precision) è una metrica di apprendimento automatico progettata per valutare gli algoritmi di rilevamento degli oggetti.

La mAP deriva da una metrica più elementare, l’Average Precision (AP), che si calcola separatamente per ogni classe. Variando la soglia di confidenza, precision e recall cambiano, ordinando le predizioni dalla più alla meno confidente e accumulando una alla volta si ottiene la curva precision-recall della classe, e l’AP ne misura l’area sottesa.

In formula, indicando con $p(r)$ la precision in corrispondenza di una recall r :

$$AP = \int_0^1 p_{\text{interp}}(r) dr \quad \text{con} \quad p_{\text{interp}}(r) = \max_{r' \geq r} p(r')$$

dove p_{interp} è la curva resa monotona decrescente, così da eliminare le piccole oscillazioni della precision.

La mAP è semplicemente la media delle AP calcolate sulle N classi del dataset:

$$mAP = \frac{1}{N} \sum_{c=1}^N AP_c$$

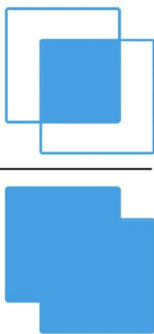
L’AP descrive quindi le prestazioni del modello su una singola classe, mentre la mAP le riassume in un unico valore per l’intero modello.

Negli articoli scientifici relativi al rilevamento di oggetti, è possibile imbattersi in abbreviazioni come mAP_{50} o mAP_{95} . In breve, questa notazione indica la soglia IoU utilizzata per stabilire se una predizione è corretta durante il calcolo delle AP. La mAP_{50} usa una sola soglia pari a 0,5, mentre la notazione estesa $mAP_{50:95}$ indica la media delle mAP ottenute con dieci soglie comprese tra 0,5 e 0,95, a passi di 0,05:

$$mAP_{50:95} = \frac{1}{10} \sum_{t \in \{0.50, 0.55, \dots, 0.95\}} mAP_t$$

IoU

L'Intersection over Union viene calcolato dividendo la sovrapposizione tra l'annotazione prevista e quella reale per l'unione di queste due.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$


[10]

Figura 3.1: Intersection over Union

Per capire meglio, ecco alcuni esempi :

- Se impostassimo la soglia IoU a 0,9, e la precision fosse pari al 16% vorrà dire che solo 1 previsione su 6 corrisponde al punteggio;
- Con una soglia pari a 0,7, ed una precision del 65% avrò che 4 previsioni sono al di sopra di tale punteggio.
- E se la soglia fosse del 0,3, e la precision salisse al 100% vuol dire che tutte le previsioni hanno un IoU superiore a 0,3.

Pertanto, la soglia IoU può influenzare significativamente il valore medio finale della Precisione Media. Questa dipendenza introduce variabilità nella valutazione del modello. Questo è negativo, perché può verificarsi uno scenario in cui un modello ottiene buoni risultati con una determinata soglia IoU e prestazioni nettamente inferiori rispetto ad un'altra.

Per contrastare questa debolezza, si è deciso di misurare la Precisione Media per ogni classe e per ogni soglia IoU, per poi calcolare la media dei punteggi ottenuti.

[11]

3.1.3 Determinismo delle metriche

Sebbene le metriche appena descritte restituiscano valori numerici precisi, l'intero ecosistema di valutazione nasconde una natura intrinsecamente stocastica. Risulta quindi fondamentale distinguere il piano puramente matematico della misurazione da quello pratico che genera le predizioni.

Il calcolo delle metriche è rigorosamente deterministico: indici come IoU o mAP, a parità di predizioni e ground truth, restituiscono sempre l'identico risultato. L'incertezza risiede a monte, nelle complesse dinamiche di addestramento. In questa fase, l'inizializzazione dei pesi, l'applicazione del dropout, la data augmentation e la gestione asincrona dei thread sulle GPU introducono componenti casuali ineliminabili. Addestrare più volte lo stesso modello produce quindi reti con pesi finali disallineati; per questo motivo, variazioni minime nelle metriche vanno interpretate come naturali fluttuazioni statistiche, e non come prove di effettivi miglioramenti architetturali. [12]

Sorprendentemente, una quota di variabilità sopravvive anche in fase di inferenza, a pesi ormai congelati. Poiché l'aritmetica in virgola mobile non gode della proprietà associativa, l'ordine di esecuzione delle somme altera lievemente il risultato numerico finale. Questo ordine dipende da fattori puramente operativi, come la dimensione del batch size, l'hardware o i dettagli implementativi delle librerie sottostanti. La stessa documentazione ufficiale di PyTorch avverte che la totale riproducibilità tra versioni software e piattaforme diverse o tra CPU e GPU non è garantita di default, a meno di non forzare un determinismo che penalizzerebbe fortemente i tempi di calcolo. [13] [14]

Infine, va considerato che i punteggi di confidenza dei moderni detector risultano tipicamente mal calibrati e tendono a un'eccessiva sicurezza, non riflettendo la reale probabilità statistica di correttezza della predizione. Per questi motivi, i valori restituiti dalle metriche non vanno intesi come garanzie assolute sul comportamento del modello in produzione, bensì come stime statistiche empiriche, strettamente ancorate a una singola esecuzione dell'addestramento e al limitato campione di immagini utilizzato. [15]

3.2 Script

Di seguito sono definite le operazioni necessarie, e specifiche per l'addestramento del modello base per la object detection YOLO26n.

3.2.1 Dataset

Il dataset utilizzato per il training di questo specifico modello YOLO è composto da una serie di immagini contenenti vari capi d'abbigliamento. Nello specifico, le annotazioni di questo dataset sono state generate tramite un ulteriore modello di segmentazione, chiamato SAM3, che verrà approfondito in seguito.

Per la fase di inferenza è stato invece impiegato un dataset più ridotto. Poiché quest'ultimo comprendeva inizialmente alcune immagini già presenti nel set di training, è stata effettuata un'operazione di pulizia dei dati: tali immagini sono state rimosse dal dataset di addestramento, garantendo così una maggiore affidabilità nella valutazione qualitativa del modello.

3.2.2 Training

Inizializzazione

```
1 import ultralytics
2 from ultralytics import YOLO
```

Elaborazione

```
1 model = YOLO("yolo26n.pt")
2
3 trainArgs = {
4     'data' : "dataset/dataset.yaml",
5     'epochs' : 100,
6     'imgsz' : 640,
7     'batch' : 64,
8     'device' : 0,
9     'name' : "detection",
10    'workers' : 4,
11    'patience' : 10,
12    'plots' : True,
13 }
14
15 model.train(**trainArgs)
```

Parametri

Dei parametri utilizzati nell'addestramento, i più rilevanti per le prestazioni sono :

- epochs : numero di epoche
- batch size : numero di immagini alla volta che gli passo
- device : hardware che voglio utilizzare
- workers : numero di "lavoratori" CPU che mi aiutano in questo processo

3.2.3 Risultati

Alla fine della fase di addestramento, verranno prodotte delle statistiche che ci serviranno per la valutazione del nostro modello, cioè la precision, recall, F1 score e l'mAP.

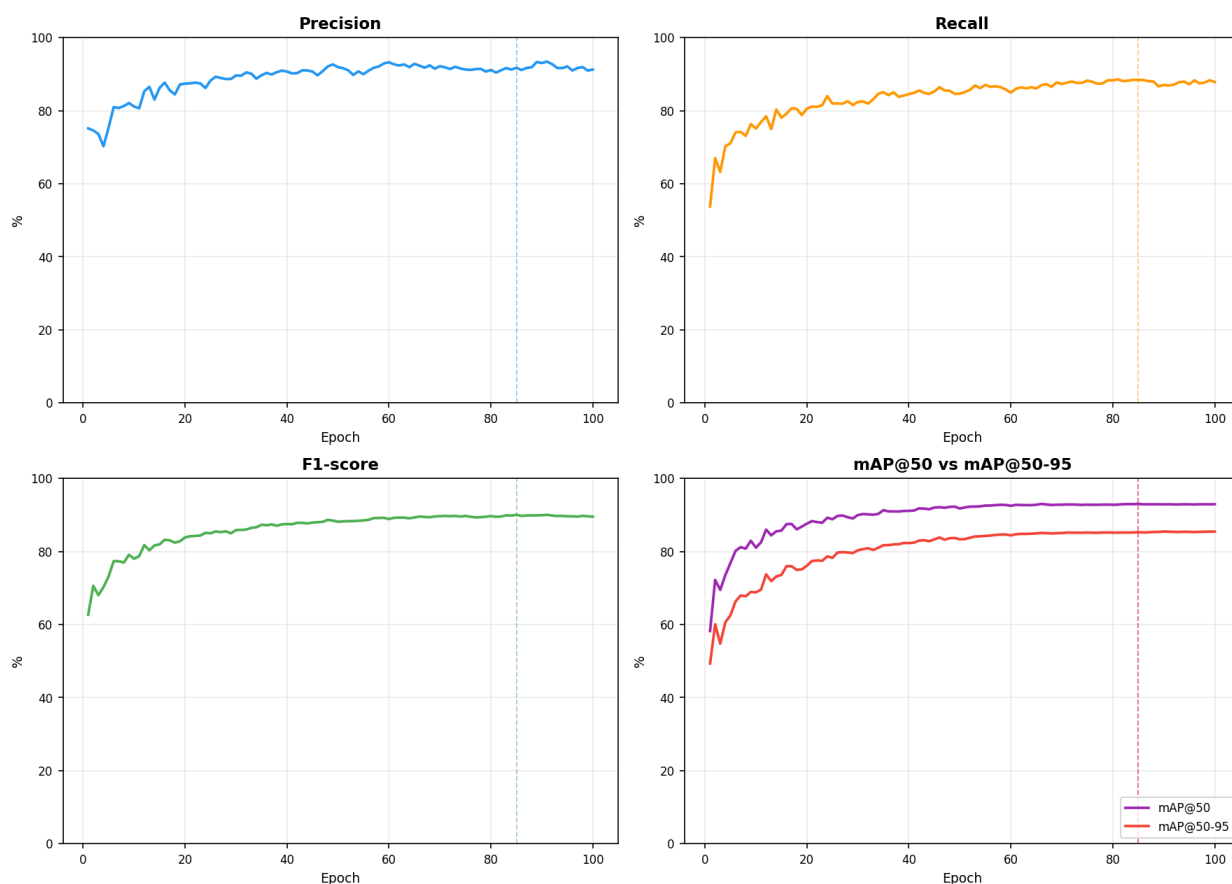
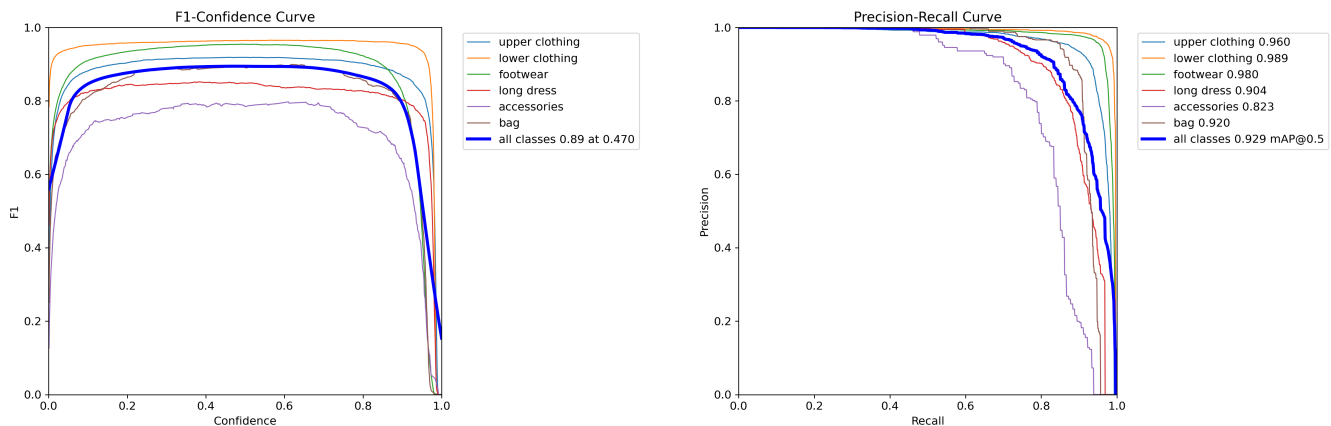


Figura 3.2: statistiche

Possiamo notare come inizialmente i grafici siano più rumorosi, e come invece più si va avanti nel numero di epoche, più la curva si stabilizzerà raggiungendo il suo plateau.

Confidence

È il livello minimo di confidenza entro il quale il modello considera l'oggetto come valido. Questo è ottenuto andando a guardare il risultato dell'IoU e controllando che sia al sopra di questa soglia minima, che solitamente è impostata di default al 50%.



Nel grafico a sinistra possiamo vedere che il punto massimo della curva F1, la quale rappresenta il punto ottimale in cui la soglia di confidence fornisce il miglior compromesso tra precision e recall, è di 0.89 ottenuto con confidence 0.470.

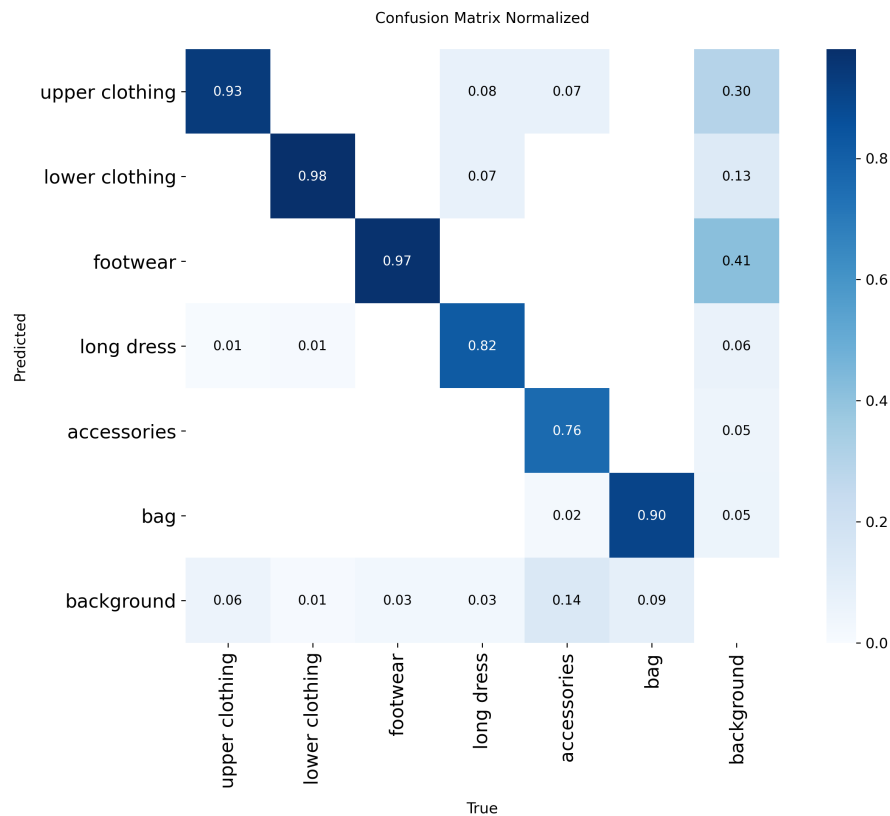
Quindi il miglior bilanciamento tra la precision e la recall si otterrà impostando una confidence, cioè un limite di soglia, pari al 47%. Questo grafico è molto importante per scegliere il valore di confidence da fornire in fase di inferenza, dove appunto si minimizzano il più possibile i fenomeni di falso positivo (precision) e falso negativo (recall).

In quello a destra, invece mostra la relazione fra precision e recall, con un valore di mAP50 pari a 0.929, e indica la media della precision agli intervalli di recall per tutte le classi, alla soglia di IoU di 0.5.

In sostanza evidenzia il bilanciamento tra la capacità del modello di identificare in maniera corretta oggetti (precision) e la capacità di trovarli tutti (recall) per i vari livelli di soglia di confidenza.

Confusion matrix

La confusion matrix mostra come il modello classifica le classi su cui è stato addestrato.



Il grafico può essere riassunto con alcune delle sue classi come :

- Predicted "upper clothing" | True "upper clothing" :
- il modello ha identificato correttamente il 93% della categoria "upper clothing".
- Predicted "background" | True "accessories" :
- il 14% degli accessori è stato classificato come "background" (falso negativo).
- Predicted "footwear" | True "background" :
- il 41% del "background" è stato classificato come tale (vero negativo).
- Predicted "background" | True "background" :
- nessun "background" è stato erroneamente classificato come capo d'abbigliamento (falso positivo assente).

3.2.4 Inferenza

Dopo aver addestrato il modello con il nostro dataset, è possibile procedere con l'inferenza su altre immagini per verificare la presenza di alberi. Importante è soprattutto fare l'inferenza su un dataset diverso da quello utilizzato nella fase di training, così da ottenere dei risultati più veritieri.

È importante assicurarsi che le immagini in input siano dello stesso formato di quelle preparate per il training, in questo caso 640x640, devono quindi rispettare la risoluzione e la composizione dei channels RGB.

Qui di seguito è dettagliata la funzione Python per l'inferenza con YOLO di Ultralytics, tramite la libreria OpenCV utilizzata in questo caso per lavorare con le immagini in input e output.

```

1 def run(modelPath, imagesDir, outputDir="predictions", conf=0.25):
2     model = YOLO(modelPath)
3     Path(outputDir).mkdir(parents=True, exist_ok=True)
4
5     for imgPath in Path(imagesDir).iterdir():
6         img = cv2.imread(str(imgPath))
7         if img is None:
8             continue
9
10        for result in model(img, conf=conf):
11            for box in result.boxes.xyxy.cpu().numpy().astype(int):
12                x1, y1, x2, y2 = box
13                cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
14
15        cv2.imwrite(f"{outputDir}/{imgPath.stem}_result.jpg", img)

```

Questa funzione implementa la fase di inferenza del modello YOLO26 su un insieme di immagini. Dato il percorso dei pesi addestrati e quello della cartella di input, il modello viene caricato tramite la libreria Ultralytics e applicato iterativamente a ciascuna immagine, con una soglia di confidenza configurabile che filtra le predizioni meno affidabili.

Per ogni immagine vengono estratte le bounding box predette, sovrapposte all'immagine originale e il risultato viene salvato in una cartella di output dedicata, consentendo così una verifica qualitativa delle detection prodotte dal modello.

4. SAM3

Il Segment Anything Model 3 (SAM 3) è un modello foundation di computer vision sviluppato da Meta e integrato nella libreria Ultralytics, progettato per compiti di rilevamento, segmentazione di istanze (instance segmentation) e tracciamento di oggetti all'interno di sequenze video. Rispetto alle versioni precedenti (SAM 1 e SAM 2), le quali limitavano la segmentazione a un singolo oggetto individuato tramite prompt visivi puntuali (come coordinate o riquadri delimitatori), SAM 3 introduce il paradigma della Promptable Concept Segmentation (PCS). Grazie a questo approccio a vocabolario aperto (open-vocabulary), il modello è in grado di identificare e segmentare contemporaneamente tutte le istanze appartenenti a un determinato concetto visivo presente nella scena. Tali concetti possono essere specificati tramite descrizioni in linguaggio naturale (es. "gatto a strisce"), esemplari visivi di riferimento o combinazioni di entrambi.

Dal punto di vista architetturale, SAM 3 disaccoppia la fase di riconoscimento da quella di localizzazione mediante l'uso di un token di presenza globale. Il sistema separa nettamente il modulo di rilevamento — basato su un'architettura di tipo DETR con fusion encoder — dal tracker video con memoria ereditato da SAM 2. Addestrato sul vasto dataset SA-Co (comprendente oltre 5 milioni di immagini e 1,4 miliardi di maschere), SAM 3 raggiunge prestazioni allo stato dell'arte con un raddoppio dell'efficienza nella segmentazione per concetto e un tempo medio d'inferenza di circa 30 ms per immagine. [16]

4.1 Generazione del Dataset

Inizialmente si era partiti da un insieme primario di immagini privo delle annotazioni necessarie per l'addestramento dei modelli di object detection. Date le elevate capacità di generalizzazione e accuratezza di SAM 3, il modello è stato impiegato come strumento di annotazione automatica (pseudo-labeling) per la costruzione del dataset finale.

Nello specifico, la funzionalità PCS ha consentito di estrarre le regioni d'interesse fornendo in input al modello le immagini accompagnate da prompt testuali descrittivi delle varie categorie di capi d'abbigliamento da identificare. L'elaborazione ha prodotto per ciascuna immagine le relative maschere di segmentazione per le istanze rilevate.

Poiché l'obiettivo finale risiedeva nell'addestramento del modello di object detection YOLO26 — il quale richiede in input delimitatori rettangolari (bounding box) anziché maschere poligonali — si è proceduto alla conversione sintetica delle annotazioni: a partire dai contorni di ciascuna maschera generata da SAM 3, sono state calcolate le coordinate estreme per ricavare i bounding box corrispondenti. Tale procedura ha permesso di trasformare efficacemente un processo di segmentazione di istanze in un dataset strutturato per l'addestramento di modelli di object detection.

4.2 Script

Lo script realizza un annotatore automatico: prende in ingresso una cartella di immagini prive di etichette e produce in uscita un dataset di object detection in formato YOLO, senza alcun intervento manuale. Il ruolo di SAM3 è quello di annotatore zero-shot: non viene addestrato né adattato al dominio, ma viene interrogato in linguaggio naturale con i nomi delle classi che si vogliono ottenere. L'annotazione umana, che è la voce di costo dominante nella costruzione di un dataset, viene sostituita dalla definizione di una lista di prompt testuali.

Un punto concettuale importante è che SAM3 è un modello di segmentazione, quindi restituisce maschere a livello di pixel. La detection non è quindi un output nativo del modello, ma una proiezione: la maschera è la rappresentazione intermedia da cui si ricava la bounding box. Questo spiega perché lo stesso identico flusso possa alimentare, senza costi aggiuntivi di inferenza, anche un dataset di segmentazione semantica.

4.2.1 Librerie

```
1 import shutil
2 import torch
3
4 from pathlib import Path
5 from PIL import Image
6 from transformers import Sam3Model, Sam3Processor
```

4.2.2 Configurazione

Tutto ciò che definisce il dataset è dichiarativo: le cartelle di input e output, l'elenco delle classi e le soglie. L'ordine di CLASSES definisce implicitamente il class_id numerico usato nelle annotazioni YOLO.

CLASS_PROMPTS introduce una distinzione utile da sottolineare: la classe del dataset non coincide necessariamente con il prompt dato al modello. Categorie generali come "accessories" sono concetti astratti che il modello riconosce male; interrogandolo invece con termini concreti (hat, cap) la resa migliora sensibilmente, e tutte le maschere trovate confluiscono comunque nell'unica classe di destinazione.

```
1 IMAGES_DIR = Path("dataset/images")    # immagini senza etichette
2 OUTPUT_DIR = Path("output/detection")  # dataset generato
3
4 CLASSES = [
5     "upper clothing",
6     "lower clothing",
7     "shoes",
8     "long dress",
9     "accessories",
10    "bag",
11 ]
12
13 CLASS_PROMPTS = {
14     "accessories": ["hat", "cap", "scarf"],
15     "bag": ["handbag", "backpack"],
16 }
17
18 SCORE_THRESHOLD = 0.5 # confidenza minima delle maschere restituite da SAM3
19 IOU_MERGE_THRESHOLD = 0.5 # IoU >= soglia --> stesso oggetto
20 MIN_AREA_RATIO = 0.001 # scarta le maschere piccole
21 DEVICE = "cuda"
22 DTYPE = torch.float16 # tipo di dato
```

4.2.3 Main

Prepara la struttura delle cartelle e carica il modello una sola volta fuori dal ciclo e itera sulle immagini. Ogni immagine annotata produce una copia nella cartella `images/` e il corrispondente file di etichette in `labels/`; le immagini in cui non viene rilevato nulla vengono semplicemente saltate e non entrano nel dataset.

```

1 def main():
2     images_out = OUTPUT_DIR / "images"
3     labels_out = OUTPUT_DIR / "labels"
4
5     for folder in (images_out, labels_out):
6         folder.mkdir(parents=True, exist_ok=True)
7
8     (OUTPUT_DIR / "classes.txt").write_text("\n".join(CLASSES) + "\n")
9
10    # caricamento del modello
11    model = Sam3Model.from_pretrained("facebook/sam3",
12                                     torch_dtype=DTYPE).to(DEVICE).eval()
13    processor = Sam3Processor.from_pretrained("facebook/sam3")
14
15    for path in sorted(IMAGES_DIR.iterdir()):
16        label_file = labels_out / f"{path.stem}.txt"
17
18        if RESUME and label_file.exists():
19            continue
20
21        image = Image.open(path).convert("RGB")
22        masks, labels = annotate(model, processor, image)
23        if not masks:
24            continue
25
26        width, height = image.size
27        shutil.copy2(path, images_out / path.name)
28        label_file.write_text("\n".join(to_yolo(masks, labels, width, height)) +
29                                   "\n")
30
31    print(f"{path.name}: {len(masks)} oggetti annotati")

```

4.2.4 Risultati



Mettiamo a confronto la fotografia di catalogo in ingresso (a sinistra) e il risultato della segmentazione (a destra).

L'input è un'immagine a figura intera di 1850x2500 pixel, priva di annotazioni: il sistema non riceve né riquadri né alcuna indicazione sul numero di capi presenti. In uscita ogni capo riconosciuto è evidenziato da una maschera a livello di pixel, con una tinta distinta, il riquadro che lo delimita e l'etichetta della categoria assegnata. Nell'esempio vengono individuati cinque capi distribuiti su quattro categorie: la giacca in pelliccia (upper clothing, confidenza 0.961), i pantaloni (lower clothing, 0.984), la borsa (bag, 0.969) e le due calzature, segmentate come istanze separate (shoes, 0.941 e 0.930). Le maschere seguono il profilo del capo e non il riquadro che lo contiene: la giacca esclude il volto e le mani, e le regioni assegnate a capi diversi non condividono alcun pixel. Le due calzature occupano rispettivamente lo 0,6% e lo 0,4% della superficie dell'immagine, al di sotto della soglia d'area minima prevista per i capi principali: vengono conservate grazie al limite ridotto applicato alle categorie di piccola dimensione (calzature, borse e copricapi).

4.2.5 Ottimizzazioni

Durante l'utilizzo di questo modello, si è notato un carico di computazione abbastanza importante, ottenendo delle tempistiche alquanto pessime, in media secondi ad immagine processata. Perciò questo tipo di problematica è stata risolta applicando principalmente due ottimizzazioni.

1. Riutilizzo dell'encoder visivo.

SAM3, come tutti i modelli della famiglia SAM, è composto da un encoder visivo pesante e da una testa di decodifica leggera condizionata dal prompt. L'encoder dipende solo dall'immagine, non dal testo: rieseguirlo per ogni prompt significherebbe ripetere inutilmente la parte più costosa del modello. Qui l'embedding visivo viene calcolato una sola volta per immagine, e poi passato a tutte le chiamate successive.

Con la configurazione attuale - sei classi per un totale di nove prompt - significa un solo passaggio dell'encoder invece di nove: il costo per immagine diventa una codifica visiva più nove decodifiche leggere, e cresce in modo quasi piatto all'aumentare del numero di classi. Questa è di fatto l'ottimizzazione che rende praticabile l'annotazione di dataset con molte categorie.

2. Inferenza in fp16.

Il modello è caricato in mezza precisione. Su GPU dotate di Tensor Core il guadagno è netto e, trattandosi di sola inferenza senza accumulo di gradienti, l'output è di fatto equivalente a quello in fp32.

4.3 Conclusioni

L'utilizzo di questo modello è stato fondamentale per il nostro lavoro: le sue capacità di segmentazione hanno permesso di automatizzare parte dell'annotazione, riducendo i tempi di preparazione e aumentando la qualità dei dati. A differenza di un semplice bounding box che introduce rumore includendo lo sfondo, la segmentazione isola esclusivamente i pixel dell'oggetto di interesse, generando dataset più precisi e coerenti a partire dalle sole immagini.

Come mostrato nei capitoli successivi, il modello otterrà prestazioni superiori rispetto agli altri analizzati, confermando l'efficacia di questo approccio pur in presenza di lievi disallineamenti metodologici nel confronto.

5. Segformer

Negli ultimi anni, questo settore ha vissuto una profonda rivoluzione guidata dall'introduzione dei modelli Transformer. Originariamente concepiti per l'elaborazione del linguaggio naturale, questi modelli sono stati adattati con successo all'analisi delle immagini grazie al pionieristico Vision Transformer (ViT), presentato nel 2020. Il ViT ha dimostrato che è possibile analizzare le immagini scomponendole in una griglia di piccoli tasselli (patch) e trattandoli in modo simile alle parole di una frase. [17]

Sebbene il ViT abbia raggiunto risultati eccezionali, presentava limiti critici per la segmentazione: restituiva mappe a bassa risoluzione spaziale e richiedeva risorse di calcolo troppo elevate per immagini di grandi dimensioni. Per superare questi ostacoli e rendere la segmentazione altamente efficiente, la ricerca ha prodotto architetture di nuova generazione come SegFormer. [18]

SegFormer è un framework progettato per bilanciare efficienza e precisione senza appesantire la potenza di calcolo. Il cuore di questa architettura è il Mix Transformer (MiT), un encoder strutturato in modo gerarchico. A differenza del ViT originale, il MiT estrae informazioni a scale diverse, riducendo progressivamente la risoluzione dell'immagine mentre ne arricchisce il significato semantico. Inoltre, il MiT elimina la necessità di assegnare coordinate di posizione fisse ai pixel, consentendo al modello di adattarsi in modo naturale a immagini di qualsiasi dimensione.

Le informazioni estratte dall'encoder vengono poi inviate a un decoder con una struttura estremamente leggera basata esclusivamente su MLP. Evitando moduli di decodifica complessi, questo design snello fonde i dettagli geometrici dei primissimi livelli con la forte comprensione contestuale degli ultimi livelli.

Nonostante SegFormer sia molto efficiente, esso opera all'interno di un cosiddetto "vocabolario chiuso": la rete è programmata per riconoscere in modo automatico solo uno specifico e limitato insieme di categorie prestabilite durante l'addestramento. L'attuale frontiera della visione artificiale si sta invece muovendo verso i modelli open-vocabulary multimodali, il cui massimo esponente è SAM 3.

SAM 3 introduce una capacità innovativa nota come "Segmentazione di Concetti basata su Prompt". Mentre SegFormer richiede etichette numeriche predefinite, SAM 3 permette all'utente di rilevare, segmentare e tracciare qualsiasi oggetto o concetto guidando il modello tramite una frase di testo, un'immagine di esempio o un click del mouse. Questa versatilità estrema non si limita alle singole foto, ma si estende nativamente al tracciamento coerente lungo i fotogrammi di un video. [19]

In conclusione, mentre architetture snelle come SegFormer rimangono molto valide per applicazioni specifiche in tempo reale e con risorse computazionali limitate, i cosiddetti Foundation Models come SAM 3 estendono le potenzialità della segmentazione verso un vocabolario illimitato.

6. VLM

6.1 Introduzione

I modelli visivo-linguistici (VLM) rappresentano una grande evoluzione nella comprensione delle immagini. In generale, il loro funzionamento si basa sulla collaborazione in sequenza di tre elementi principali: il processo inizia con un modulo visivo che "osserva" l'immagine e ne cattura i dettagli essenziali; successivamente, un traduttore converte queste informazioni visive in un formato leggibile dall'intelligenza artificiale, creando un ponte tra le immagini e le parole. Infine, entra in azione il cervello linguistico, che riceve i dati visivi tradotti assieme alle richieste testuali dell'utente, combinandoli per elaborare una risposta discorsiva e di senso compiuto. Questo flusso di lavoro integrato rappresenta un grande passo in avanti rispetto alla computer vision classica, ma è importante fare una distinzione rispetto ai modelli tradizionali ultra-specializzati, come YOLO o SAM. Essendo focalizzati su un unico obiettivo visivo, questi algoritmi tradizionali rimangono nettamente superiori in termini di precisione geometrica: riescono a localizzare e isolare gli elementi nella foto con un'accuratezza che i VLM, dovendo gestire simultaneamente la visione e l'elaborazione del linguaggio, non possono eguagliare.

La vera forza dei VLM, tuttavia, non risiede nel ritaglio geometrico perfetto, ma nella profonda comprensione semantica della scena. Mentre modelli come YOLO o SAM si limitano a individuare meccanicamente la presenza isolata all'interno della foto, un VLM utilizza il ragionamento logico per interpretare il contesto, deducendo e spiegando a parole quello che avviene all'interno dell'immagine. In questo modo, i VLM superano la rigidità dei compiti tradizionali, unendo la semplice percezione visiva alla capacità di ragionare e dialogare.

6.2 Qwen 3

Creato da Alibaba Cloud, Qwen3-VL è un modello costruito per elaborare contemporaneamente un'enorme quantità di informazioni. La sua forza sta nel riuscire a gestire testi lunghissimi mescolati a decine di immagini e video, senza mai perdere il filo del discorso. [20]

Per fare questo, utilizza due soluzioni intelligenti: la prima gli permette di avere un senso dell'orientamento eccezionale, capendo perfettamente dove si trova un oggetto nello spazio di una foto o l'esatto ordine cronologico in un video; la seconda lo aiuta a non perdersi nemmeno i dettagli visivi più piccoli, unendoli al testo in modo coerente. Grazie a queste abilità avanzate, Qwen3-VL può svolgere compiti pratici sorprendenti: è in grado di "guardare" l'interfaccia di un computer o di uno smartphone per capire come usarla, oppure può osservare lo schizzo grafico di un sito web e scrivere in automatico il codice informatico per costruirlo. [21] [22]

6.2.1 Fine-tuning

Il principio è lo stesso descritto in generale nella Sezione 2.3.3: si parte dai pesi pre-addestrati del modello e se ne prosegue l'addestramento sui nostri dati. Nel caso di Qwen3-VL, però, sia la natura del modello sia le sue dimensioni impongono alcune scelte specifiche.

Qwen3-VL è un modello puramente linguistico: non calcola coordinate geometriche, ma genera semplici stringhe di testo (token) osservando un'immagine.

Questo significa che ogni forma di addestramento applicata è, tecnicamente, l'addestramento di un modello linguistico. Non potendo utilizzare una classica funzione di errore spaziale, come la "loss di detection" tipica della computer vision, l'unica vera scelta riguarda l'origine del segnale usato per valutare e correggere le risposte del modello. Per questo motivo, sono state adottate due diverse politiche di apprendimento.

LoRA

La tecnica LoRA (Low-Rank Adaptation) nasce da una semplice intuizione: per adattare un modello serve solo una piccola correzione mirata, non una riscrittura da zero. Invece di modificare milioni di parametri originali, i quali vengono "congelati". LoRA concentra l'aggiornamento in due tabelle leggerissime, con poche migliaia di valori che agiscono come un filtro correttivo. A fine addestramento, queste tabelle vengono fuse nel modello di base, mantenendone inalterata la velocità di esecuzione.

È importante precisare che LoRA non è un obiettivo di apprendimento (non decide cosa il modello deve imparare), ma una pura tecnica di efficienza (decide quali parti aggiornare). Per questo può essere abbinata a qualsiasi metodo di addestramento, compresi quelli descritti nei paragrafi successivi.

Cross-Entropy

Nel fine-tuning supervisionato disponiamo, per ogni immagine, della risposta esatta che vorremmo far produrre al modello.

Conoscendo in anticipo il risultato atteso, possiamo fornire al modello l'intera sequenza corretta insieme all'immagine e alla domanda, sfruttando una tecnica nota come teacher forcing. Invece di fargli generare il testo parola per parola, il modello elabora tutta la sequenza simultaneamente in un unico passaggio, calcolando quale probabilità avrebbe assegnato a ciascuna parola corretta. Da questo calcolo si ricava il margine di errore, che viene poi utilizzato in un unico passaggio a ritroso per aggiornare le matrici di LoRA.

Tuttavia, il modo in cui questo errore viene calcolato presenta un limite strutturale profondo, fondamentale per comprendere i risultati dei nostri esperimenti. Il sistema valuta le parole prodotte dal modello come un vocabolario di simboli del tutto distinti, tra i quali non esiste un concetto di vicinanza. Nella formula di base, il modello non ha modo di intuire che quei numeri rappresentano una posizione nello spazio e che, geometricamente parlando, "sbagliare di poco" è infinitamente meglio che "sbagliare di molto".

GRPO

Tramite Reinforcement Learning si confrontano i contorni disegnati dal modello con quelli reali, assegnando un punteggio da 0 a 1 che penalizza il sistema se dimentica oggetti, ne inventa di inesistenti o li inquadra in modo impreciso.

Per capire se un punteggio sia effettivamente buono, abbiamo utilizzato la tecnica GRPO (Group Relative Policy Optimization). Il principio è semplice: il modello genera diverse risposte alternative per la stessa immagine. Quelle che ottengono un punteggio superiore alla media del gruppo vengono premiate e rinforzate, mentre quelle sotto la media vengono scoraggiate.

Il prezzo da pagare per questa libertà è puramente computazionale. Poiché il modello deve materialmente generare diverse risposte parola per parola, invece di leggerne una sola in simultanea.

Risultati

Le tre configurazioni confrontate sono cumulative: Base è il modello pre-addestrato senza alcun adattamento, C-E aggiunge il fine-tuning supervisionato con cross-entropy, GRPO applica sopra quest'ultimo un ulteriore stadio di apprendimento per rinforzo guidato da una ricompensa basata sull'IoU.

	P	R	<i>F1</i>	AP₅₀	AP_{50:95}
Base	0.9020	0.9460	0.9235	0.8556	0.5746
C-E	0.9387	0.9287	0.9337	0.8878	0.5586
GRPO	0.9360	0.9332	0.9346	0.8884	0.5610

I risultati dei test mostrano due tendenze opposte: l'adattamento migliora nettamente la precisione generale riducendo i falsi positivi, ma peggiora l'accuratezza geometrica dei contorni.

Questo accade perché il sistema di addestramento non comprende la vicinanza spaziale: per il modello, sbagliare una coordinata di un pixel o di cento comporta la stessa penalità. Impara quindi molto bene cosa identificare e quanti oggetti ci sono, ma non dove tracciarne esattamente i bordi.

Inoltre, abbiamo notato che il modello adattato diventa più prudente, rischiando di ignorare qualche oggetto reale pur di non generare falsi allarmi. L'apprendimento per rinforzo ha corretto in parte questo sbilanciamento, ma non ha migliorato il disegno dei contorni, a causa di parametri di addestramento risultati troppo conservativi.

In conclusione, l'adattamento di un VLM migliora enormemente la sua capacità di riconoscere il contenuto dell'immagine, ma non affina il ritaglio spaziale. La precisione millimetrica resta un suo limite strutturale, un compito in cui i rilevatori specializzati rimangono insuperabili.

6.3 Gemma 4

Sviluppato da Google, Gemma 4 è un modello che si distingue per la sua capacità di adattarsi a situazioni diverse. La sua innovazione principale è la capacità di guardare le immagini con livelli di "attenzione" regolabili. In pratica, se il compito richiede molta precisione, il modello può analizzare l'immagine in modo minuzioso; se invece serve rapidità, come quando deve elaborare i numerosi fotogrammi di un video, può dare un'occhiata più veloce e generale. [23]

Poiché Gemma 4 è un ottimo "pensatore" ma è meno adatto a calcolare con esattezza la posizione geometrica delle cose, spesso viene fatto lavorare in squadra con programmi visivi più classici. In questa collaborazione, il programma tradizionale fa il lavoro manuale (come isolare gli oggetti nella foto), mentre Gemma 4 fa da "mente", analizzando il risultato e scrivendo un resoconto comprensibile a parole.

6.3.1 Risultati

Per verificare se una maggiore capacità di elaborazione potesse superare i limiti evidenziati, abbiamo confrontato Qwen3-VL (8 miliardi di parametri) con la versione di Gemma 4 da 31 miliardi di parametri. La scelta di impiegare modelli di sviluppatori differenti ha un fine puramente esplorativo: l'obiettivo è analizzare le potenzialità e i limiti strutturali della tecnologia VLM nel suo complesso, senza alcuna intenzione di focalizzarsi su una competizione tra produttori.

Mantenendo l'addestramento tramite cross-entropy (come fatto per Qwen3-VL), i risultati di Gemma 4 si sono rivelati inaspettatamente simili a quelli del modello più piccolo. L'aumento esponenziale dei parametri non ha prodotto alcun salto qualitativo significativo, mostrando solo variazioni marginali: una leggera diminuzione della precision a fronte di un lieve miglioramento della recall.

Questo esito conferma che il collo di bottiglia nella localizzazione non risiede nella dimensione del modello, ma nel metodo di apprendimento. Poiché la cross-entropy tratta le coordinate come semplici stringhe di testo prive di nozioni spaziali, un modello più intelligente non possiede comunque gli strumenti matematici per disegnare contorni migliori. Va tuttavia precisato che l'esperimento è stato condotto su un dataset di dimensioni ridotte; è quindi probabile che la scarsità di esempi abbia penalizzato il modello da 31 miliardi, limitandone il pieno potenziale computazionale.

In sintesi, pur tenendo conto del limite imposto dai dati, l'esperimento dimostra che scalare i parametri da 8B a 31B si rivela inefficace se la funzione di loss non è strutturalmente adatta al compito geometrico richiesto. Aumentare le dimensioni di un VLM, da solo, non basta a trasformarlo in un rilevatore spaziale di precisione.

7. Grounding DINO

Storicamente, l'Object Detection è stata dominata da reti neurali convoluzionali come YOLO. Pur avendo rivoluzionato il settore grazie all'elaborazione ultra-rapida dell'immagine in un singolo passaggio per restituire i bounding box, YOLO è un sistema Closed-Set, cioè è capace di riconoscere solo le categorie per cui è stato esplicitamente addestrato e fallisce su oggetti al di fuori di questo vocabolario. [24]

Per superare questo limite, sono nati modelli multimodali Open-Set come Grounding DINO. Il termine "grounding" indica la capacità di ancorare un concetto testuale a un'entità fisica nell'immagine. Fornendo un'immagine e una descrizione a parole, il modello estrae le caratteristiche visive e testuali tramite due Backbone dedicati, entrambi basati sulla tecnologia Transformer. Utilizzando poi meccanismi di Cross-Attention, fa "dialogare" queste fonti: l'immagine si allinea al significato delle parole e viceversa. Il risultato è la generazione di bounding box precisi per qualsiasi oggetto descritto, senza necessità di riaddestramento. [25]

Mentre Grounding DINO eccelle nel rilevamento tramite bounding box, un'altra evoluzione parallela basata sui Transformer è il Segment Anything Model (SAM). Pur condividendo tecnologie simili, i due si differenziano per obiettivi e risoluzione spaziale:

- L'obiettivo geometrico (YOLO e Grounding DINO): approccio sintetico incentrato sulle coordinate esterne, ideale per il conteggio o il tracciamento rapido nella scena.
- L'obiettivo topologico (SAM): non ci si limita a inscatolare l'oggetto, ma genera una maschera esatta che ne segue i contorni reali.

In sintesi, la Computer Vision odierna offre strumenti complementari: l'efficienza di YOLO per contesti chiusi e noti; la flessibilità open-set di Grounding DINO per localizzare concetti testuali; e la precisione al pixel di SAM per estrarre le forme nel dettaglio.

8. Test

Dopo aver visto questi modelli, possiamo vedere le varie differenze tra di loro andando a confrontarli con quelle che sono le proprietà principali per la valutazione di un modello, le quali abbiamo visto in precedenza.

Per condurre questo studio e avere un solido punto di riferimento esterno, abbiamo inizialmente selezionato un modello di terze parti. Di tale modello non possediamo dettagli specifici riguardanti l'architettura esatta, la provenienza dei dati originali o le metodologie con cui è stato precedentemente addestrato; operando in assenza di queste informazioni a priori, esso si è configurato per noi, a tutti gli effetti, come un sistema a "scatola chiusa" (black box).

8.1 Ground truth

Per poter valutare oggettivamente le prestazioni di questo strumento e degli altri algoritmi a nostra disposizione, è emersa la necessità fondamentale di definire un metro di giudizio assoluto, rigoroso e imparziale. A tale scopo, abbiamo costruito una cosiddetta ground truth (verità di base) partendo dal medesimo dataset di immagini che sarebbe stato poi sottoposto in fase di test. Nello specifico, l'intero set di dati è stato analizzato e verificato manualmente, ispezionando il campione un'immagine alla volta. Questo meticoloso processo di revisione umana ci ha permesso di generare un dataset pulito, accurato e privo di errori, che rappresenta il risultato ideale a cui ogni algoritmo dovrebbe tendere.

La ground truth è composta, per ogni immagine, da un insieme di oggetti annotati manualmente. Ogni oggetto è descritto da:

- una **classe**, scelta dalla tassonomia del dataset;
- un **bounding box**, cioè il rettangolo che racchiude l'oggetto, espresso in pixel tramite le coordinate (x_1, y_1, x_2, y_2) degli angoli opposti.

La valutazione si concentra sulle tre classi *upper clothing*, *lower clothing* e *footwear*. Il bounding box è quindi l'unità di misura comune: qualunque sia il tipo di output di un modello, questo viene sempre ricondotto a dei box prima del confronto.

Il valore strategico di questa ground truth si è rivelato cruciale per l'intera fase di testing. Essa, infatti, non è servita esclusivamente a valutare il modello black box, ma è stata impiegata come standard universale per testare e verificare le performance di svariati altri modelli che abbiamo preso in esame.

I modelli da confrontare sono molto diversi tra loro (detector, modelli di segmentazione, modelli open-vocabulary e il modello black box), per cui è necessario che le loro predizioni vengano misurate tutte nello stesso modo. Abbiamo quindi definito un unico protocollo di valutazione, indipendente dall'architettura del singolo modello: ogni modello viene eseguito sulle immagini della ground truth, le sue predizioni vengono ricondotte a un formato comune e infine confrontate con le annotazioni manuali.

8.1.1 Abbinamento tra predizioni e ground truth

Il passaggio centrale della valutazione è decidere quale predizione corrisponde a quale oggetto della ground truth. Per ogni immagine l'abbinamento avviene in modo *greedy*:

1. le predizioni vengono ordinate dalla più confidente alla meno confidente;
2. per ogni predizione, in quest'ordine, si cerca tra gli oggetti della ground truth non ancora assegnati quello con il punteggio di sovrapposizione più alto;
3. se questo punteggio supera la soglia prevista, la predizione e l'oggetto vengono abbinati e l'oggetto non può più essere assegnato ad altre predizioni; altrimenti la predizione resta senza abbinamento.

Ogni oggetto della ground truth può quindi essere abbinato a una sola predizione, e se un modello produce più box sullo stesso oggetto, solo il più confidente viene abbinato, mentre gli altri vengono considerati errori. Da notare che il punteggio di sovrapposizione usato per l'abbinamento cambia a seconda del tipo di modello.

Detection Per i modelli di detection si usa l'IoU, che come abbiamo visto misura quanto il box predetto P e il box della ground truth G coincidono.

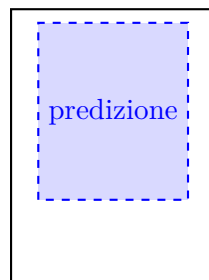
Segmentazione Per i modelli di segmentazione l'IoU risulterebbe troppo severo, visto che il box ricavato da una maschera può essere più piccolo del box annotato, ad esempio quando una parte del capo è coperta, oppure quando la maschera lo segue più da vicino del rettangolo disegnato a mano. In questi casi l'oggetto è stato trovato, ma l'IoU sarebbe basso. Per questo motivo si usa il **contenimento**, che misura quanta parte della predizione cade all'interno del box della ground truth:

$$C(P, G) = \frac{|P \cap G|}{|P|}$$

Il contenimento vale 1 quando la predizione è interamente contenuta nell'oggetto annotato, senza pretendere che ne indovini l'esatta estensione. Poiché anche un box minuscolo interno all'oggetto avrebbe contenimento pari a 1, si aggiunge un controllo sulla copertura, cioè sulla parte del box della ground truth coperta dalla predizione:

$$\text{Cov}(P, G) = \frac{|P \cap G|}{|G|}$$

Una predizione viene abbinata se $C \geq 0.5$ e $\text{Cov} \geq 0.1$.



Contenimento

Figura 8.1: Una predizione interamente contenuta nell'oggetto

8.1.2 Confusion matrix

Una volta abbinate le predizioni, i risultati di tutte le immagini vengono raccolti in una confusion matrix. In questa fase si considerano solo le predizioni con confidenza di almeno 0.5, così da valutare il modello nelle condizioni in cui verrebbe realmente usato.

Le righe della matrice rappresentano la classe reale (quella della ground truth), le colonne la classe predetta. Oltre alle classi, la matrice contiene:

- una riga **background**, per le predizioni che non sono state abbinate a nessun oggetto (il modello ha visto un oggetto dove non c'era);
- una colonna **background**, per gli oggetti della ground truth rimasti senza predizione (il modello non li ha visti);
- una colonna **non mappate**, per le predizioni con una classe estranea alla ground truth.

GT \ Pred	upper	lower	footwear	background
upper	TP	-	-	FN
lower	-	TP	-	FN
footwear	-	-	TP	FN
background	FP	FP	FP	-

Tabella 8.1: Struttura confusion matrix - colonna predizioni non mappate omessa

Ogni predizione abbinata incrementa la cella che incrocia la classe reale dell'oggetto e la classe predetta: se le due coincidono si trova sulla diagonale, altrimenti il modello ha localizzato correttamente l'oggetto ma ne ha sbagliato la classe. Ogni predizione non abbinata finisce nella riga background e ogni oggetto mancato nella colonna background.

8.1.3 Metriche

Dalla confusion matrix si ricavano, per ogni classe c , i tre conteggi di base:

- **TP** (veri positivi): la cella sulla diagonale, cioè gli oggetti di classe c trovati e classificati correttamente;
- **FP** (falsi positivi): il resto della colonna c , cioè le predizioni di classe c che corrispondono a un oggetto di un'altra classe o a nessun oggetto;
- **FN** (falsi negativi): il resto della riga c , cioè gli oggetti di classe c classificati come un'altra classe o non trovati affatto.

Un errore di classe viene quindi contato due volte: come falso positivo per la classe predetta e come falso negativo per la classe reale. A partire da questi conteggi si calcolano precision e recall, così come definite nel capitolo su YOLO, e l'F1 score, che ne è la media armonica:

$$Precision = \frac{TP}{TP + FP} \quad Recall = \frac{TP}{TP + FN} \quad F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

La precision indica quanto ci si può fidare delle predizioni del modello, la recall quanti degli oggetti presenti il modello riesce a trovare, e l'F1 score riassume entrambe in un unico valore, che è alto solo se lo sono tutte e due.

Average Precision. Precision e Recall dipendono dalla soglia di confidenza scelta. Per avere una misura che non dipenda da questa scelta viene calcolata anche l'AP₅₀ per ogni classe:

1. si raccolgono tutte le predizioni della classe su tutte le immagini, questa volta con una soglia di confidenza molto bassa (0.001);
2. ogni predizione viene marcata come corretta o errata tramite l'abbinamento descritto in precedenza;
3. ordinando le predizioni per confidenza decrescente e accumulando i conteggi, si ottiene la curva precision-recall del modello;
4. l'AP₅₀ è l'area sotto questa curva, resa prima monotona decrescente.

Questa è la stessa Average Precision da cui deriva la mAP introdotta nel capitolo su YOLO (Sezione 3.1.2), calcolata con una soglia di IoU pari a 0,5, o di contenimento per i modelli di segmentazione.

8.2 Confronto tra modelli

L'aver calcolato le statistiche di tutti i modelli sulla medesima base di dati verificata a mano ci ha consentito di metterli a confronto diretto con il modello esterno di riferimento. Tutti i modelli vengono valutati con le stesse soglie, lo stesso criterio di abbinamento per tipo di modello e le stesse formule: utilizzando lo stesso identico metro di paragone per ogni sistema, abbiamo garantito un confronto totalmente equo (fair comparison). In questo modo, possiamo affermare con certezza che le differenze prestazionali riportate nei grafici delle sezioni successive riflettono le reali capacità predittive dei singoli modelli, e non il modo in cui sono state misurate né eventuali bias derivanti dal dataset di valutazione.

In ciascuno dei grafici seguenti vengono riportate, per le tre classi *upper clothing*, *lower clothing* e *footwear*, le metriche di precision, recall e F1 score dei modelli presi in esame, affiancate a quelle del modello black box. Le metriche sono calcolate secondo il protocollo descritto precedentemente, considerando le predizioni con confidenza di almeno 0.5.

8.2.1 Dataset di addestramento

I modelli proposti sono stati addestrati a partire da due dataset differenti:

- **HF**: dataset pubblico di object detection per l'abbigliamento disponibile su Hugging Face [26], composto da circa 90.000 immagini. Le sue sette categorie originali (*top*, *outer*, *bottom*, *dress*, *shoes*, *bag*, *hat*) sono state ricondotte alle tre classi della valutazione, e le maschere per la segmentazione sono state generate con SAM3;
- **StyleMix**: circa 16.800 immagini provenienti dal catalogo StyleMix. A differenza dell'altro dataset, non si tratta di una raccolta pubblica reperibile in rete, ma di materiale di provenienza privata, fornito direttamente da un brand di abbigliamento e quindi non liberamente distribuibile. Le immagini sono le fotografie utilizzate dal brand per il proprio catalogo, che vanno a rappresentare il dominio applicativo reale a cui il sistema è destinato. Il catalogo è stato fornito privo di annotazioni ed è stato etichettato automaticamente tramite SAM3, con la procedura descritta nella sezione successiva (Sezione 8.2.2).

Dei modelli proposti, ad eccezione di SAM3, usato senza addestramento, e del black box, tutti i modelli riportati nei grafici sono addestrati sul dataset StyleMix. Fa eccezione SegF_HF, che condivide l'architettura con SegF_StyleMix ma è addestrato sul dataset HF.

8.2.2 Annotazione con SAM3

Per ottenere le annotazioni necessarie all'addestramento, SAM3 è stato impiegato come annotatore automatico, estendendo la procedura descritta nel capitolo dedicato in modo da generare, con un'unica inferenza per immagine, sia il dataset di detection che quello di segmentazione. Per ogni immagine il procedimento è il seguente:

1. l'encoder visivo di SAM3 viene eseguito una sola volta e il suo risultato viene riutilizzato per tutti i prompt testuali;
2. il modello viene interrogato con un prompt per ciascuna classe (*upper clothing*, *lower clothing*, *footwear*, *long dress*, *accessories*, *bag*), mantenendo solo le maschere con confidenza di almeno 0.5; per le classi più generiche si usano più prompt concreti, ad esempio *hat*, *cap* e *scarf* per gli accessori;
3. le maschere della stessa classe che si sovrappongono con $\text{IoU} \geq 0.5$ vengono considerate lo stesso capo e fuse in un'unica area;
4. se viene rilevato un abito lungo, questo ha la precedenza e le classi *upper clothing* e *lower clothing* non vengono cercate, così da non spezzare un capo intero in due parti.

Le maschere ottenute alimentano poi i due dataset:

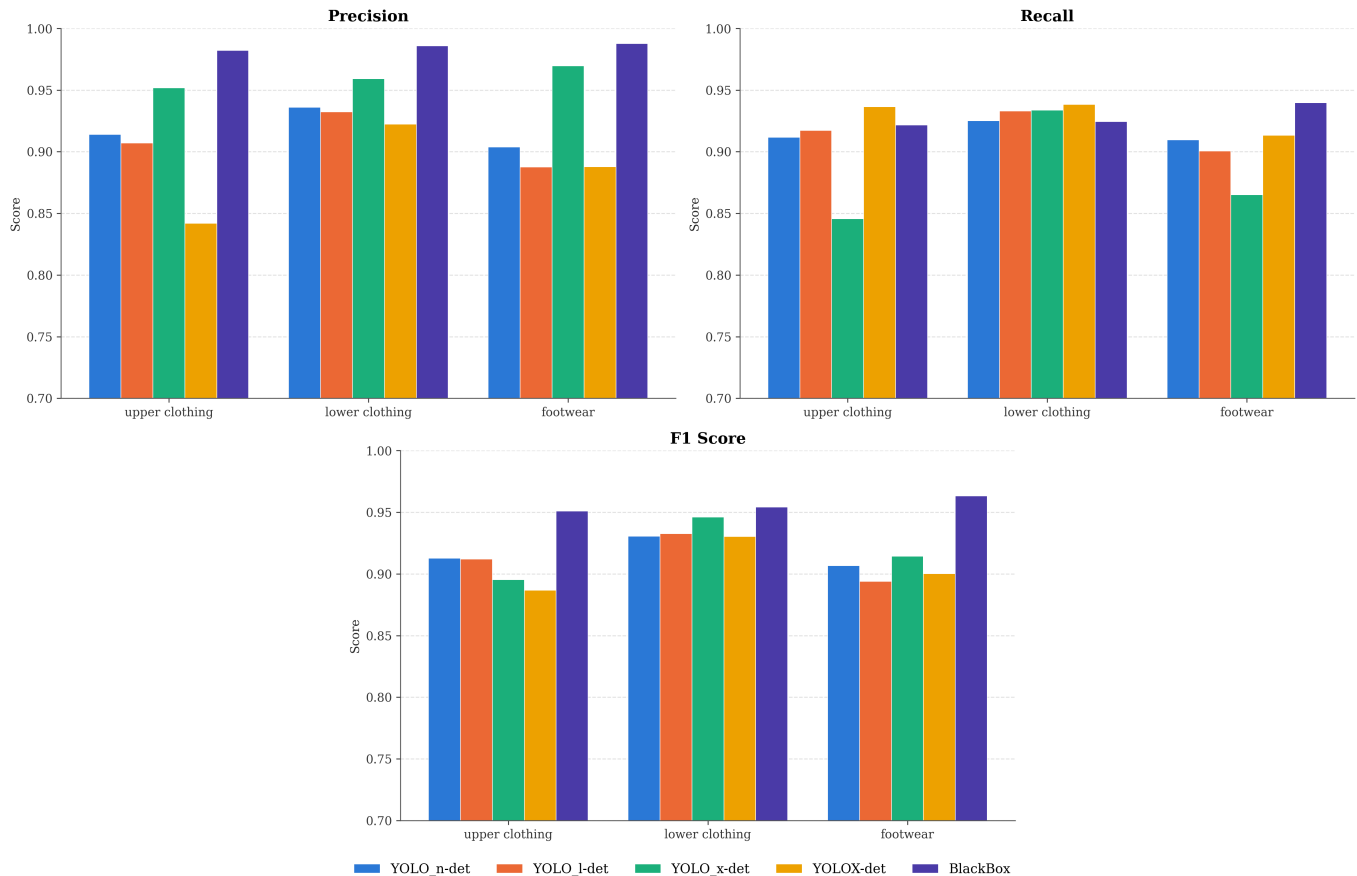
- per la **detection**, ogni maschera viene convertita nel suo box circoscritto in formato YOLO, scartando i box con un'area inferiore allo 0.1% dell'immagine;
- per la **segmentazione**, le maschere vengono "dipinte" su un'unica mappa in cui ogni pixel contiene l'identificativo della classe, con 0 per lo sfondo. Le maschere più grandi vengono disegnate per prime, così che un capo piccolo, come una scarpa, non venga coperto da uno più grande in caso di sovrapposizione.

La procedura è stata applicata in modo diverso sui due dataset. Per StyleMix, privo di annotazioni, SAM3 ha generato entrambi i dataset a partire da 16.834 immagini. Per HF, che dispone già di box annotati, il dataset di detection deriva dalle annotazioni originali ricondotte alle nuove classi, mentre SAM3 è stato usato per generare le maschere di segmentazione, non presenti nel dataset, su 89.630 immagini. In entrambi i casi il tempo medio è di circa 2.2 secondi per immagine.

Questi tempi sono resi possibili dalle ottimizzazioni introdotte: su un benchmark di 50 immagini, l'uso della precisione a 16 bit e il riutilizzo dell'encoder visivo riducono il tempo di inferenza da circa 18.3 a 1.3 secondi per immagine, con un'accelerazione di circa $13.6\times$ e lo stesso numero di capi rilevati.

8.2.3 YOLO detection

Il primo confronto riguarda le diverse varianti di YOLO addestrate per il task di object detection.

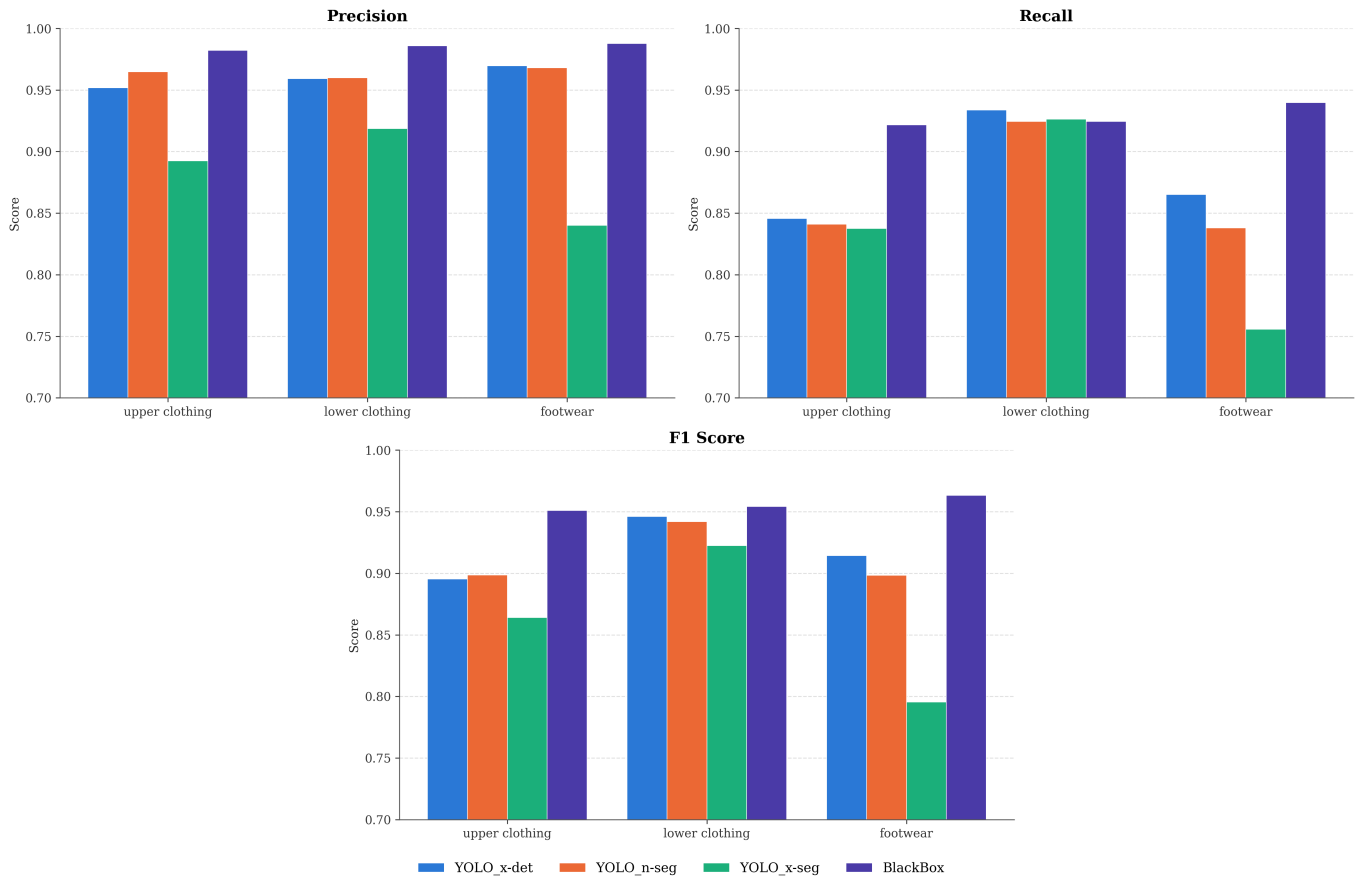


Come si può osservare, il modello black box mantiene la precision più alta su tutte e tre le classi, con valori vicini a 0.98, e solo YOLO_x-det riesce ad avvicinarsi a questo risultato, in particolare su *footwear*. YOLO_x-det paga però questa precision con una recall sensibilmente più bassa su *upper clothing* e *footwear*, risultando più conservativo: produce poche predizioni errate ma tralascia più oggetti. YOLOX-det mostra il comportamento opposto, con la recall più alta sulle prime due classi, superiore anche a quella del black box, ma con la precision più bassa, che su *upper clothing* scende a circa 0.84.

YOLO_n-det e YOLO_l-det si collocano in una posizione intermedia, con precision e recall più bilanciate; da notare che la versione large non supera mai in modo significativo la versione nano. In termini di F1 score, YOLO_x-det risulta il più vicino al black box su *lower clothing* e *footwear*, mentre su *upper clothing* viene superato dalle versioni nano e large.

8.2.4 YOLO segmentation

Il secondo confronto mette a paragone le varianti di YOLO per la segmentazione, con il miglior modello di detection individuato in precedenza, YOLO_x-det.



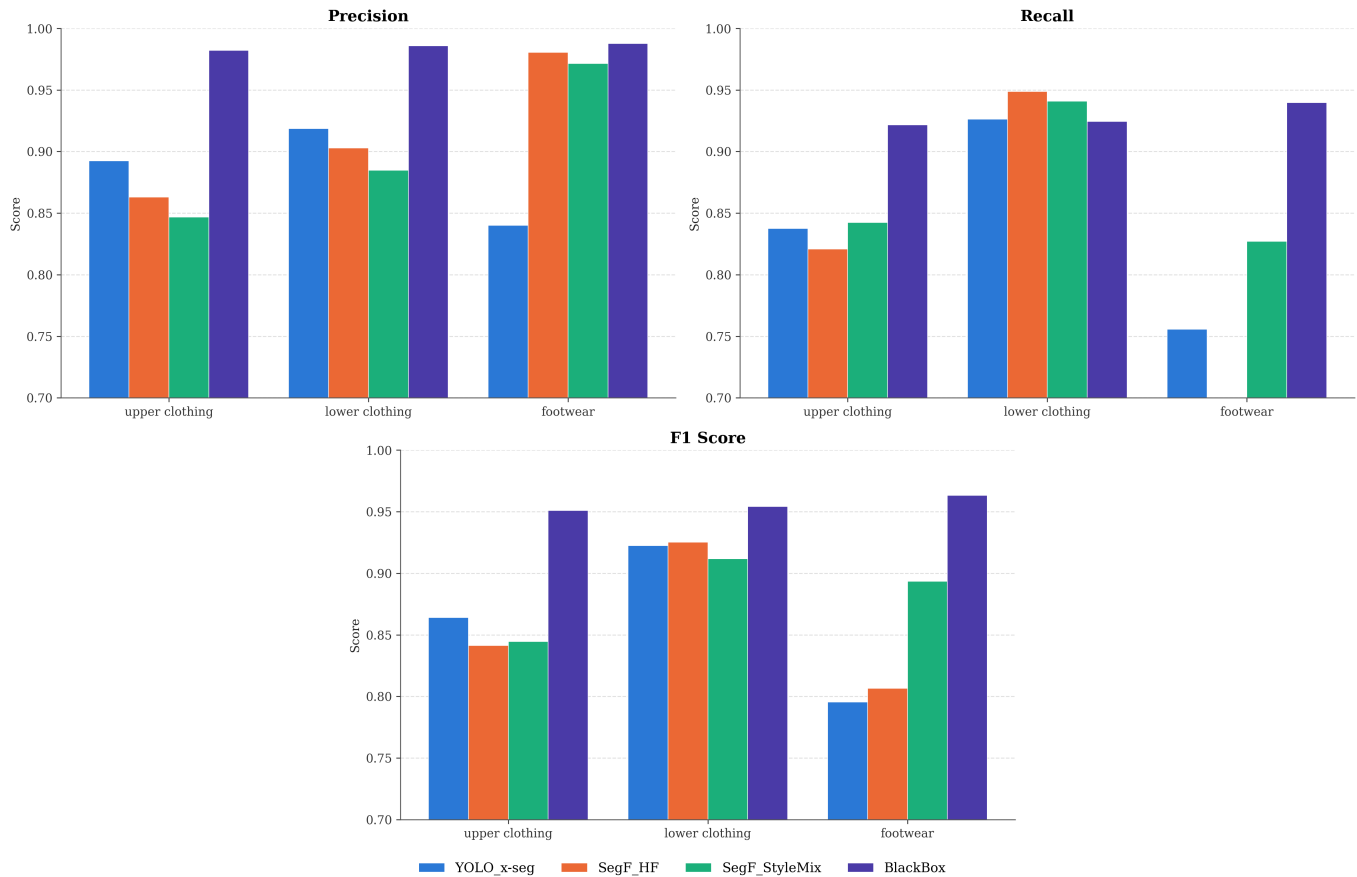
Dal grafico emerge che YOLO_n-seg ottiene risultati molto simili a YOLO_x-det: la precision è sostanzialmente la stessa, mentre la recall è di poco inferiore, soprattutto su *footwear*. Ne deriva un F1 score quasi identico sulle prime due classi e leggermente più basso sulla terza; anche in questo caso, quindi, la versione più piccola del modello si dimostra competitiva.

YOLO_x-seg ha invece prestazioni complessivamente inferiori, in modo evidente su *footwear*, dove la precision si ferma a circa 0.84 e la recall scende intorno a 0.75, portando l'F1 score sotto 0.80. Le calzature, essendo oggetti piccoli e spesso parzialmente coperti, rappresentano la classe più difficile per questo modello.

Tutti e tre i modelli hanno infine una recall su *upper clothing* intorno a 0.84, circa otto punti al di sotto del black box, mentre su *lower clothing* risultano allineati a quest'ultimo.

8.2.5 SegFormer

Il terzo confronto riguarda i modelli basati su SegFormer: SegF_HF e SegF_StyleMix, confrontati con YOLO_x-seg.

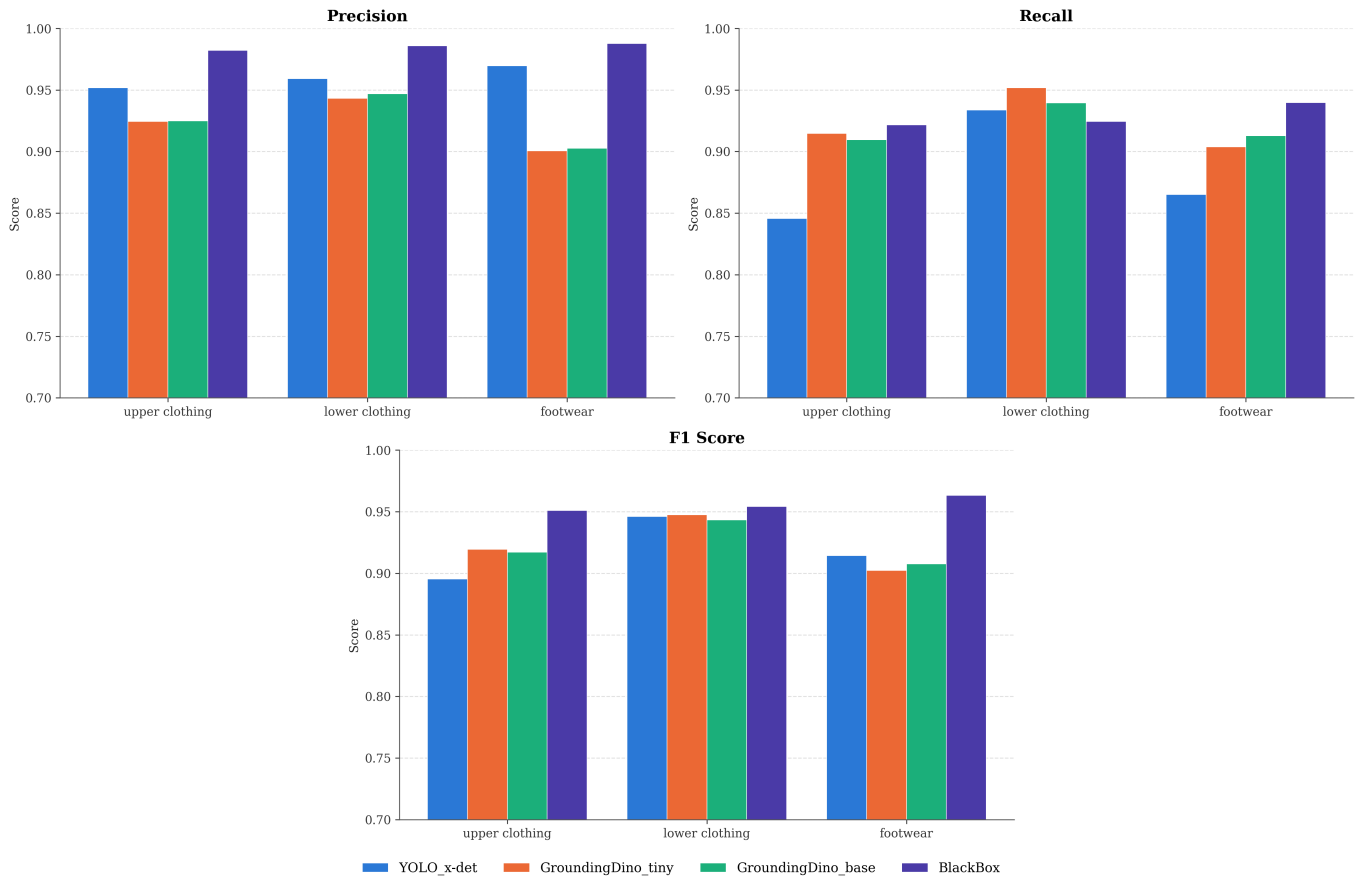


In figura si nota come i modelli SegFormer abbiano una precision molto elevata su *footwear*, vicina a quella del black box e nettamente superiore a YOLO_x-seg. A questa corrisponde però una recall bassa, che nel caso di SegF_HF scende al di sotto della scala del grafico: il modello individua con sicurezza le calzature che trova, ma ne tralascia una parte consistente. SegF_StyleMix migliora sensibilmente questo aspetto, portando la recall a circa 0.83 e ottenendo su questa classe l’F1 score più alto tra i modelli di segmentazione, pur restando al di sotto del black box.

Su *upper clothing* i modelli SegFormer hanno una precision inferiore a YOLO_x-seg e una recall simile, per cui è quest’ultimo a ottenere l’F1 score migliore. Su *lower clothing*, invece, raggiungono la recall più alta del grafico, superando anche il black box, ma la precision più bassa mantiene l’F1 score in linea con quello di YOLO_x-seg.

8.2.6 GroundingDINO

Il quarto confronto riguarda i modelli open-vocabulary GroundingDino_tiny e GroundingDino_base, confrontati con YOLO_x-det.

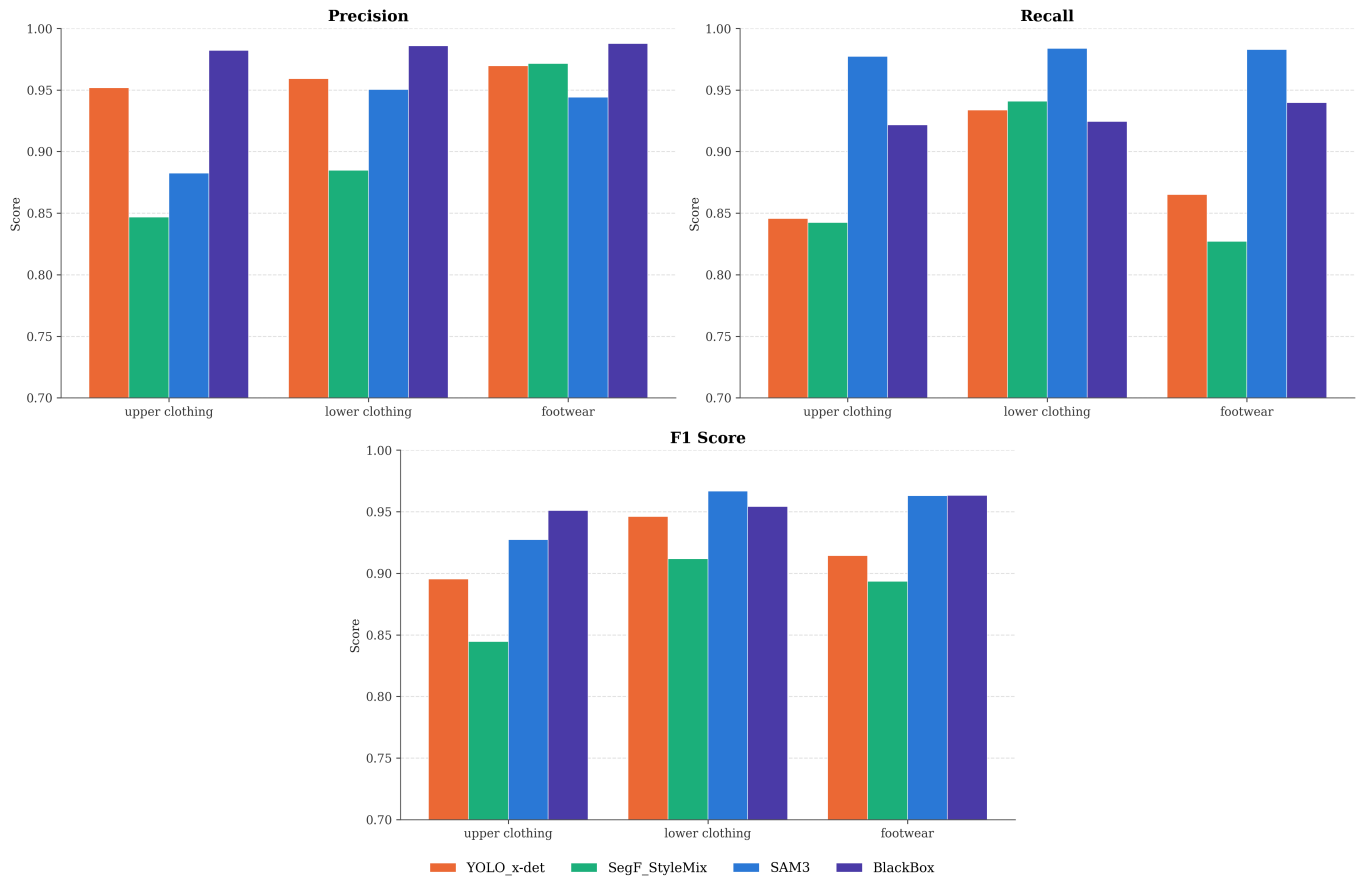


Come mostrato, entrambe le versioni di GroundingDINO hanno una precision inferiore a YOLO_x-det su tutte le classi, con un distacco più ampio su *footwear*, dove scendono a circa 0.90 contro 0.97. In compenso, la recall è più alta su tutte le classi: su *upper clothing* passa da circa 0.85 a 0.91, e su *lower clothing* GroundingDino_tiny raggiunge circa 0.95, superando anche il black box.

Ne risulta un F1 score migliore su *upper clothing*, sostanzialmente equivalente su *lower clothing* e leggermente inferiore su *footwear*, dove il calo di precision non viene compensato del tutto dalla recall. Le differenze tra la versione tiny e la versione base sono minime, nell'ordine di un punto percentuale o meno: anche in questo caso, un modello più grande non porta vantaggi significativi.

8.2.7 SAM3

Infine, abbiamo confrontato SAM3 con i migliori modelli emersi dai confronti precedenti: YOLO_x-det e SegF_StyleMix.



Qui emerge che SAM3 raggiunge la recall più alta in assoluto, superiore a 0.97 su tutte le classi e quindi anche a quella del black box, che si ferma tra 0.92 e 0.94. Il vantaggio è particolarmente evidente rispetto a YOLO_x-det e SegF_StyleMix su *upper clothing* e *footwear*, dove questi ultimi si fermano intorno a 0.83–0.87.

La sua precision resta invece inferiore a quella del black box, con un distacco di circa dieci punti su *upper clothing* e di circa quattro sulle altre due classi. SAM3 e il black box presentano quindi profili di errore complementari: il primo tende a produrre qualche falso positivo in più, il secondo a tralasciare qualche oggetto.

In termini di F1 score, SAM3 supera il black box su *lower clothing*, lo eguaglia su *footwear* e rimane leggermente al di sotto su *upper clothing*. Ottiene inoltre l’F1 score migliore del confronto su tutte le classi, risultando il modello più vicino al riferimento esterno e l’unico in grado di raggiungerne o superarne le prestazioni su almeno una classe.

8.3 Dai modelli al sistema

Il percorso seguito fin qui ha avuto un obiettivo preciso, cioè capire quale fosse lo strumento più adatto a riconoscere i capi d'abbigliamento all'interno di una fotografia. Dopo aver introdotto i concetti alla base delle reti neurali e del loro addestramento, abbiamo analizzato e addestrato modelli di natura molto diversa, i quali per poterli confrontare in modo equo abbiamo costruito una ground truth verificata a mano e un protocollo di valutazione comune, che ha permesso di misurarli tutti con lo stesso metro e di metterli a confronto con un modello esterno di riferimento.

Il risultato di questo confronto è stato netto. SAM3 si è rivelato il modello con la recall più alta, in grado di individuare la quasi totalità dei capi presenti nelle immagini, e l'unico a raggiungere o superare il modello "black box" in termini di F1 score su almeno una classe. A questo si aggiunge un vantaggio che le metriche non misurano direttamente, SAM3 non richiede alcun addestramento sul nostro dominio, perché viene interrogato in linguaggio naturale, e può quindi cercare qualunque tipo di capo semplicemente cambiando il prompt testuale.

Riconoscere un capo, però, non è di per sé un risultato utile per l'utente finale. Un modello di segmentazione restituisce maschere e riquadri, cioè informazioni che descrivono una singola immagine, ma non ci dice nulla sui rapporti tra i prodotti di un catalogo, quali capi vengono indossati insieme, quali stili si abbinano, quale articolo potrebbe completare un outfit. Per rispondere a domande di questo tipo serve una struttura capace di raccogliere ciò che il modello osserva nelle singole fotografie e di trasformarlo in conoscenza organizzata e interrogabile.

Le fotografie indossate di un catalogo contengono un'informazione implicita molto preziosa: ogni immagine mostra un abbinamento reale, scelto da chi ha curato il catalogo. Utilizzando SAM3 per isolare i singoli capi presenti in ciascuna fotografia, e riconducendo poi ogni capo all'articolo corrispondente tramite una ricerca per somiglianza visiva, è possibile trasformare queste co-occorrenze in un grafo, in cui i prodotti sono i nodi e gli abbinamenti osservati sono le relazioni tra di essi.

Il lavoro svolto nei capitoli precedenti non rappresenta quindi un fine a sé stante, ma la base su cui poggia un sistema concreto, cioè l'analisi dei modelli ha permesso di scegliere in modo motivato il componente più affidabile, e il protocollo di valutazione ha dato una misura oggettiva di quanto ci si possa fidare delle sue predizioni. Nel prossimo capitolo vedremo come questo componente venga integrato in un'architettura GraphRAG, che affianca un database vettoriale per la ricerca per somiglianza e un database a grafo per la gestione delle relazioni, dando vita a un sistema in grado di rappresentare e interrogare gli abbinamenti tra i prodotti del catalogo.

9. Graph RAG

Sebbene la RAG tradizionale sia eccellente nel recuperare documenti basandosi sulla similarità semantica tramite database vettoriali, si scontra frequentemente con una fisiologica perdita di contesto. I modelli vettoriali, infatti, faticano nel multi-hop reasoning, ovvero nella capacità di estrarre e dedurre relazioni tra concetti distanti all'interno di un vasto corpus documentale. Questo porta spesso a risposte incomplete o fuorvianti, specialmente quando le connessioni tra le informazioni non sono esplicitate nel testo.

Per arginare questo problema, è stato formalizzato il concetto di GraphRAG, un paradigma che sostituisce o affianca i vettori ai Knowledge Graph (KGs), permettendo così di mappare in modo esplicito le entità e le loro interrelazioni. Tuttavia, l'implementazione originale di GraphRAG si rivela fortemente "LLM-centrica", introducendo nuove criticità architetturali. Demandare a un Large Language Model l'intera costruzione del grafo e la sua successiva interrogazione — ad esempio costringendolo a tradurre a runtime il linguaggio naturale in query Cypher — espone il sistema al rischio di allucinazioni sintattiche, inconsistenze ontologiche e a un'impennata insostenibile dei costi di inferenza su larga scala.

La soluzione ingegneristica proposta supera questi colli di bottiglia integrando un database vettoriale (Qdrant) con un database a grafo (Neo4j), facendoli comunicare attraverso la condivisione di identificatori univoci.

9.1 Ciclo di vita

Il ciclo di vita del dato all'interno di questo sistema si divide in due macro-fasi. Durante l'inserimento dei dati (Ingestion), i testi grezzi vengono elaborati — tramite LLM o tecniche di Named Entity Recognition — per estrarne un'ontologia strutturata di entità e relazioni, che viene archiviata nel database a grafo Neo4j. Parallelamente, gli stessi documenti vengono convertiti in embedding vettoriali e memorizzati in Qdrant. In questo modo, il sistema si assicura che la medesima informazione sia esplorabile sia da un punto di vista puramente semantico sia strutturale.

La vera sinergia tra le due tecnologie si esprime nella fase di recupero e generazione (Retrieval & Generation). Quando l'utente pone una domanda, il sistema non cerca di tradurla direttamente in linguaggio per i grafi, ma esegue prima una veloce ricerca vettoriale su Qdrant. Questo passaggio permette di isolare in modo efficiente i frammenti di testo semanticamente più pertinenti. Una volta ottenuti gli ID di questi documenti, il sistema li utilizza come punto di ingresso per interrogare Neo4j: il database a grafo recupera le reti relazionali associate a quegli ID, espandendo il contesto con i nodi adiacenti e le loro connessioni esplicite.

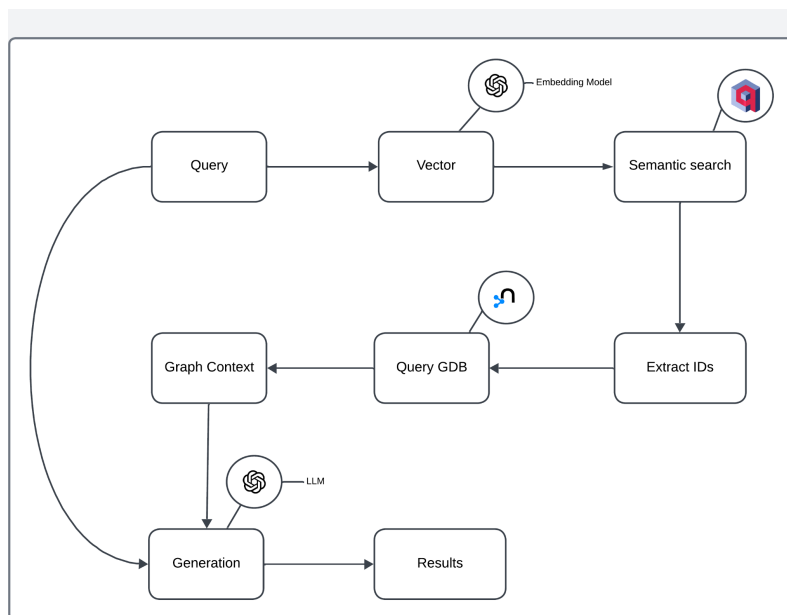


Figura 9.1: grafo RAG

Il risultato finale è un contesto ibrido, denso e deterministico, che viene fornito all'LLM per la generazione della risposta. In un'ottica di architettura del software, il valore centrale di questo approccio per una tesi di informatica risiede nel principio di disaccoppiamento delle responsabilità. Qdrant gestisce agilmente il matching semantico, Neo4j garantisce il rigore del contesto relazionale e l'LLM viene sollevato dal ruolo inappropriato di "motore di interrogazione", tornando a fungere esclusivamente da generatore linguistico. Questo paradigma ibrido abbate i costi computazionali, azzerà gli errori di querying dinamico e garantisce un'affidabilità delle risposte notevolmente superiore rispetto agli approcci monolitici. [27]

9.2 Database vettoriali

I database vettoriali rappresentano un'evoluzione fondamentale per l'Intelligenza Artificiale moderna. Essi sono progettati per memorizzare, indicizzare e interrogare dati rappresentati sotto forma di embedding, ovvero vettori numerici ad alta dimensionalità generati da modelli di machine learning. A differenza dei tradizionali motori di ricerca basati su keyword matching, i vector database operano attraverso la ricerca per similarità spaziale (es. similarità del coseno, distanza euclidea o prodotto interno). Due concetti semanticamente affini si troveranno vicini all'interno dello spazio vettoriale, indipendentemente dai termini specifici utilizzati per descriverli.

Nel contesto della nostra architettura, Qdrant ricopre l'incarico cruciale di motore di Semantic Search. Il suo compito è elaborare dati originariamente non strutturati (es. le descrizioni in linguaggio naturale dei prodotti) e mapparli vettorialmente. Quando il sistema riceve una query, Qdrant agisce come primo livello di filtraggio: isola in tempi dell'ordine dei millisecondi i frammenti informativi o i prodotti che presentano la maggiore vicinanza semantica alla richiesta dell'utente. Qdrant eccelle nel gestire l'ambiguità del linguaggio naturale, individuando il "punto di ingresso" ideale per la ricerca, ma, per sua natura, non possiede la consapevolezza strutturale di come tali informazioni siano collegate tra loro in un dominio complesso.

9.3 Database a Grafo

Per sopperire alla mancanza di contesto relazionale dei vettori, entrano in gioco i database a grafo. Questa tecnologia basa la propria architettura sulla teoria dei grafi, abbandonando il rigido schema a tabelle dei database relazionali per modellare l'informazione attraverso nodi (le entità fisiche o logiche), archi (le relazioni esplicite che collegano le entità) e proprietà. Il paradigma a grafo eccelle laddove il valore dell'informazione risiede primariamente nelle interconnessioni tra i dati.

Nel nostro sistema, Neo4j costituisce l'ossatura logica e relazionale dell'applicazione. A valle del processo di estrazione (operato da SAM3), Neo4j immagazzina l'ontologia del dominio: istanzia i prodotti come nodi e crea un reticolo denso di archi che ne definiscono categorie, compatibilità, specifiche tecniche e logiche di accoppiamento. Il principale vantaggio prestazionale di Neo4j è l'elaborazione Index-Free Adjacency: le operazioni di attraversamento del grafo avvengono in tempo reale, poiché ogni nodo mantiene puntatori fisici diretti ai nodi adiacenti. Ciò rende Neo4j lo strumento perfetto per il multi-hop reasoning, ovvero la capacità di estrarre percorsi logici complessi con latenza minima, indipendente dalle dimensioni totali del database.

9.4 Integrazione di SAM3

Come discusso in precedenza in merito all'architettura ibrida, il modello SAM3 svolge un ruolo cruciale all'interno del sistema, fungendo da motore d'inferenza principale per la fase di popolamento (o ingestion) del database a grafo, che in questa implementazione è Neo4j. Nello specifico, le avanzate capacità di analisi di SAM3 vengono impiegate per processare i dati in ingresso ed estrarre, con un elevato grado di precisione, tutte le entità e le caratteristiche salienti pertinenti al dominio applicativo in esame.

Questo processo di estrazione mirata permette di tradurre informazioni eterogenee in un'ontologia ben definita e pronta per l'archiviazione strutturata. Una volta isolati, gli elementi estratti vengono mappati e memorizzati all'interno dell'istanza di Neo4j: i prodotti di interesse vengono istanziati sotto forma di nodi, mentre le loro peculiarità, i metadati e le reciproche interdipendenze vanno a costituire gli archi (relazioni). Questa mappatura sistematica e deterministica porta alla generazione di un catalogo prodotti completo e semanticamente interconnesso, sfruttando a pieno i vantaggi di flessibilità e modellazione offerti da Neo4j rispetto ai tradizionali database relazionali.

La strutturazione dell'informazione all'interno di un property graph come Neo4j apre a scenari di elaborazione e interrogazione particolarmente avanzati. In fase di ricerca (retrieval), il sistema è in grado di avvalersi della topologia della rete, permettendo di navigare le relazioni esplicite — anche attraversando nodi multipli — per rispondere a query complesse con latenze estremamente ridotte. Inoltre, l'adozione di Neo4j si rivela fondamentale per l'implementazione di logiche di "accoppiamento significativo". Analizzando i percorsi, le adiacenze e i gradi di separazione tra i vari nodi, il sistema può dedurre connessioni semantiche non banali tra molteplici prodotti. Ciò abilita lo sviluppo di funzionalità di alto livello, come i sistemi di raccomandazione context-aware, capaci di suggerire combinazioni di articoli basate su affinità strutturali e logiche di business codificate direttamente all'interno della rete relazionale del grafo.

9.5 Pipeline

La pipeline utilizzata trasforma le fotografie indossate di un catalogo di abbigliamento in un grafo di abbinamenti. Per ogni prodotto si seleziona una fotografia a figura intera; un modello di segmentazione guidato da testo isola il capo in vendita e gli altri capi indossati; ogni capo viene ricondotto a un articolo del catalogo attraverso filtri di colore e forma e una ricerca per somiglianza visiva; infine le co-occorrenze osservate vengono scritte in un grafo a proprietà.

I dati di partenza sono l'elenco dei prodotti, con codice, titolo e immagini, e le fotografie indossate di ciascun prodotto. Il risultato è un grafo in cui i capi sono nodi e le co-occorrenze nelle fotografie sono archi.

9.5.1 Architettura

La pipeline è composta da quattro stadi eseguiti in sequenza dove ogni stadio legge da disco il risultato del precedente, e questo permette di ripetere uno stadio senza rifare quelli a monte. I due stadi più costosi sono quelli di segmentazione e riconoscimento, eseguiti entrambi da servizi remoti: la segmentazione da un worker su GPU, il riconoscimento da un servizio di ricerca visiva.

Selezione della fotografia

Nel catalogo preso in considerazione, contenente migliaia di prodotti sono state tenute le singole fotografie indossate, e poi è stato applicato un filtro andando che escludeva:

- prodotti composti da più capi venduti insieme, come i set, per i quali non esiste un singolo capo da riconoscere;
- le fotografie tagliate a metà, o più in generale quelli non a figura completa;
- le fotografie che non ritraggono una sola persona a figura intera.

Per ogni fotografia selezionata poi si prepara un insieme ordinato di nomi del capo in vendita, dal più specifico al più generico: il titolo con il sostantivo tradotto in inglese, il solo sostantivo tradotto e una categoria generica (per esempio *upper clothing*). La traduzione è necessaria perché il modello di segmentazione è addestrato su testo in inglese. Il nome generico serve anche a gestire i termini ambigui, cioè dove una parola che indica due capi diversi viene tradotta secondo il significato più frequente, e il nome generico copre l'altro, perché viene provato solo quando i primi due hanno fallito.

9.5.2 Segmentazione

Come spiegato anche precedentemente viene utilizzato SAM 3 per la fase di segmentazione, un modello che accetta come prompt una frase testuale e restituisce, per ogni istanza trovata, un riquadro, una maschera e un punteggio di confidenza.

Doppio passaggio

La parte costosa dell'inferenza è la codifica visiva dell'immagine, che viene calcolata una sola volta e riusata per tutti i prompt, in due giri:

1. **Capo in vendita:** si provano i nomi del prodotto in ordine, fermandosi al primo che produce una rilevazione valida. Se nessuno funziona, il capo viene dichiarato non trovato, senza ripiegare su parole generiche, visto che un ripiego produrrebbe un ritaglio indistinguibile da un successo.
2. **Altri capi:** si prova un vocabolario fisso di frasi, ricondotte a un insieme ridotto di etichette. Per lo stesso tipo di capo si usano più frasi, perché nessuna le trova tutte, *skirt*, per esempio, riconosce gonne che *lower clothing* non rileva.

Frase del vocabolario	Etichetta
upper clothing, coat, bikini top	upper clothing
lower clothing, skirt, midi-skirt	lower clothing
dress	dress
footwear, boot	shoes
headwear	hat
bag, belt, scarf, underwear	uguale alla frase

Tabella 9.1: Il vocabolario del secondo giro e le etichette in uscita

9.5.3 Filtri di qualità

Le rilevazioni del modello passano attraverso sei filtri prima di diventare ritagli.

Confidenza. Si scartano le rilevazioni con punteggio inferiore a una soglia τ_s , più bassa per le calzature, che il modello riconosce con confidenza minore.

Area. Si scartano i riquadri troppo piccoli, che corrispondono a dettagli come un bottone, e quelli che coprono quasi tutta l'immagine. Calzature, borse e copricapi sono piccoli per natura e hanno un limite inferiore più basso.

Occupazione. Misura quanto il capo riempie il proprio ritaglio, il numero di pixel della maschera M , diviso per il quadrato del lato maggiore del riquadro di larghezza w e altezza h :

$$\text{occ} = \frac{|M|}{\max(w, h)^2} \quad (9.1)$$

La misura diminuisce in entrambi i casi in cui un ritaglio diventa illeggibile, quando è quasi vuoto e quando è lungo e sottile. Si richiede $\text{occ} \geq \tau_{\text{occ}}$, con una soglia bassa per i capi grandi e una alta per calzature e accessori, per i quali un ritaglio è utile solo se l'oggetto si riconosce a colpo d'occhio.

Sovrapposizione. Frasi diverse possono rilevare lo stesso capo. Due riquadri A e B sono considerati lo stesso capo quando l'intersezione sull'unione supera una soglia τ_{IoU} , e si conserva quello col punteggio più alto:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \quad (9.2)$$

Un solo capo inferiore. Sotto una gonna il modello rileva anche i collant. Si conserva una sola rilevazione con etichetta *lower clothing*, quella con il punteggio più alto.

Coerenza dei vestiti. Su un completo in due pezzi la sagoma è continua e anche la frase *dress* produce una rilevazione; viceversa, su un vestito le frasi per i capi superiori e per quelli inferiori ne rilevano le due metà. Il filtro sfrutta un'informazione nota, cioè il capo in vendita; chi indossa la gonna o la maglia in vendita non indossa anche un vestito, a meno che il capo in vendita non sia un capospalla, che si porta sopra un abito. Nel primo caso le rilevazioni *dress* vengono scartate; nel secondo decide la geometria, attraverso la misura di contenimento di un riquadro a in un riquadro b :

$$\text{dentro}(a, b) = \frac{|a \cap b|}{|a|} \quad (9.3)$$

Se nel riquadro del vestito sono contenuti, oltre una soglia τ_{in} , sia un capo superiore sia uno inferiore, si tratta di un due pezzi e si scarta il vestito; se ne è contenuto uno solo, quello è metà del vestito e viene scartato.

Rimozione del capo in vendita

Il secondo giro rileva anche il capo in vendita, chiamato con la sua parola generica, e questo va rimosso per non abbinare un capo a se stesso. Si confronta il ritaglio del capo in vendita con quello di ciascun capo del secondo giro, dopo averli ridotti alla stessa piccola risoluzione su fondo uniforme. La somiglianza è il complemento della differenza assoluta media fra i n valori di colore delle due immagini:

$$s(a, b) = 1 - \frac{1}{255n} \sum_{i=1}^n |a_i - b_i| \quad (9.4)$$

Si rimuove il capo più somigliante, se supera una soglia τ_{sim} . Poiché i due ritagli provengono dalla stessa fotografia, quando rappresentano lo stesso capo sono quasi identici, e un confronto pixel per pixel è sufficiente. Ogni ritaglio è restituito come immagine con canale alfa, in cui la maschera rende trasparente tutto ciò che non appartiene al capo.

9.5.4 Riconoscimento a catalogo

Ogni ritaglio deve essere ricondotto a un articolo del catalogo. Il riconoscimento è una sequenza di tre operazioni, dove le prime due restringono lo spazio di ricerca, e la terza sceglie l'articolo.

Colore e forma

Il colore dominante del ritaglio viene espresso nella stessa terminologia usata dal catalogo. Prima della misura la trasparenza viene riempita con un colore che nessun capo presenta, così che lo sfondo non venga scambiato per il capo. Se la porzione di ritaglio effettivamente misurata è troppo piccola, si cerca senza colore, dove un colore sbagliato escluderebbe l'articolo corretto.

La forma, cioè la categoria del capo, si ottiene con una ricerca per vicinanza (*k-nearest neighbours*) in una raccolta di riferimento di capi etichettati, dove il ritaglio viene trasformato in un vettore, e la categoria viene ereditata dal vicino più simile. La categoria viene scartata quando il vicino è troppo lontano o quando fra i vicini più prossimi compaiono troppe categorie diverse.

Ricerca

La ricerca a catalogo è ibrida: interroga più rappresentazioni di ogni articolo (fotografia del prodotto, fotografia indossata, testo) e ne fonde le classifiche con la *Reciprocal Rank Fusion*. Il punteggio fuso indica la posizione in classifica ma non la somiglianza; come somiglianza si usa la similarità coseno del canale visivo.

Lo stesso ritaglio viene cercato in sei modi, che variano un fattore alla volta, e si sceglie l'articolo con la somiglianza sim_k più alta:

$$c^* = \arg \max_{k \in \{1, \dots, 6\}} \text{sim}_k \quad (9.5)$$

Una ricerca filtrata per categoria che non restituisce risultati viene ripetuta senza la categoria, perché una categoria sbagliata esclude l'articolo corretto.

	Immagine cercata	Fotografie del catalogo	Filtro
1	ritaglio	indossate	zona, colore, categoria
2	ritaglio	indossate	zona, colore
3	ritaglio	indossate	zona, categoria
4	ritaglio	prodotto	zona, colore, categoria
5	capo di riferimento	indossate	zona, colore, categoria
6	capo di riferimento	prodotto	zona, colore, categoria

Tabella 9.2: Strategie di ricerca

La scelta del massimo non è neutrale. Le strategie 2 e 3 rimuovono una parte del filtro e cercano in un insieme che contiene quello della strategia 1, quindi la loro somiglianza non può essere inferiore; le strategie 5 e 6 confrontano un'immagine pulita con immagini pulite e tendenzialmente tenderanno a valori più alti.

Ripiego per le calzature

Il catalogo contiene poche calzature rispetto a quelle che compaiono nelle fotografie. Per i ritagli di calzature, se la somiglianza a catalogo è inferiore a una soglia τ_{cat} , si adotta il capo di riferimento trovato durante il riconoscimento della forma, purché sia anch'esso una calzatura e la sua somiglianza superi una soglia τ_{rif} . Le due somiglianze provengono da sistemi diversi e non sono confrontabili, quindi il ripiego interviene per soglia e non per confronto. Il capo di riferimento può appartenere a un altro marchio e non essere acquistabile.

9.5.5 Modello dei dati

Il grafo è un grafo a proprietà con due tipi di nodo e tre tipi di arco:

```
(Prodotto)-[:HA_OUTFIT]->(Outfit)-[:CONTIENE]->(Prodotto)
(Prodotto)-[:ABBINATO_A]->(Prodotto)
```

Un nodo **Prodotto** rappresenta un articolo, identificato dal suo codice: un capo che compare in più fotografie resta un solo nodo. Un nodo **Outfit** rappresenta per l'appunto un outfit completo. Ogni outfit ha un proprietario, il capo in vendita, collegato con **HA_OUTFIT**, e contiene tutti i capi visibili, proprietario compreso, collegati con **CONTIENE**. L'arco **ABBINATO_A** riassume tutti gli outfit, andando direttamente dal capo in vendita a ogni capo visto nell'outfit completo.

Il nodo **Outfit** risolve un problema preciso, con il solo **ABBINATO_A** che aggrega gli outfit e perde l'informazione su quali capi fossero indossati insieme, chiedendo che cosa abbinare a un capo si otterrebbero la gonna di una fotografia e le scarpe di un'altra, due capi mai visti sulla stessa persona; passando per l'outfit si ottiene invece un look completo effettivamente indossato.

Elemento	Attributi	Significato
Prodotto	code	codice dell'articolo
	product_id	presente solo sui capi la cui fotografia è stata elaborata
	zona	etichetta del ritaglio che ha riconosciuto il capo, presente solo sui capi visti in fotografie altrui
Outfit	product_id	identificativo del capo proprietario della fotografia
CONTIENE	somiglianza	somiglianza del ritaglio all'articolo, assente verso il proprietario
ABBINATO_A	volte	numero di osservazioni della coppia
	lift	lift della coppia, equazione
	somiglianza	somiglianza migliore fra le osservazioni
HA_OUTFIT	$\emptyset$	$\emptyset$

Tabella 9.3: Attributi del grafo

9.5.6 Costruzione

La costruzione legge i risultati di ogni prodotto e produce una riga per ogni capo osservato, nella forma (outfit, proprietario, capo, zona, somiglianza). Da queste righe derivano sia gli outfit sia gli archi diretti. Tre regole selezionano le righe:

- un capo entra solo se la sua somiglianza supera la soglia della sua fonte, τ_{cat} per il catalogo e τ_{rif} per il riferimento;
- delle calzature se ne conserva una sola per outfit completo, la più somigliante, perché i due ritagli di scarpe mostrano lo stesso paio da due angolazioni e possono essere ricondotti ad articoli diversi;
- un capo con lo stesso codice del proprietario viene escluso.

Per ogni coppia (p, c) si calcolano il numero di osservazioni n_{pc} , il numero di righe del proprietario n_p , il numero di fotografie in cui compare il capo n_c e il numero totale di righe N . Il lift confronta la frequenza osservata della coppia con quella attesa sotto l'ipotesi di indipendenza:

$$\text{lift}(p, c) = \frac{P(p, c)}{P(p) P(c)} = \frac{n_{pc} N}{n_p n_c} \quad (9.6)$$

La scrittura nel database è stata ottimizzata per lotti: ogni istruzione riceve l'intero elenco delle righe e lo scorre lato server. Nodi e archi vengono creati in modo idempotente, per cui rieseguire la costruzione sugli stessi dati produce lo stesso grafo. Poiché il database può contenere dati di altre fonti, la costruzione non cancella mai i nodi dei prodotti: prima di riscrivere rimuove soltanto le strutture che essa stessa genera, cioè gli outfit e gli archi di abbinamento.

9.5.7 Rappresentazione grafica

Il risultato completo può essere controllato tramite la dashboard di gestione di Neo4j, la quale ci permette di visualizzarlo graficamente.

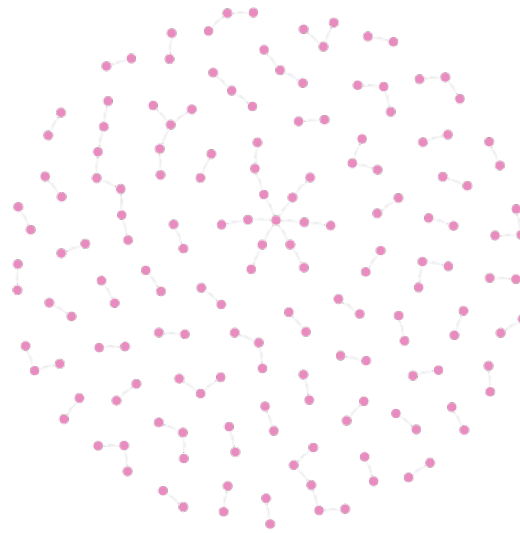
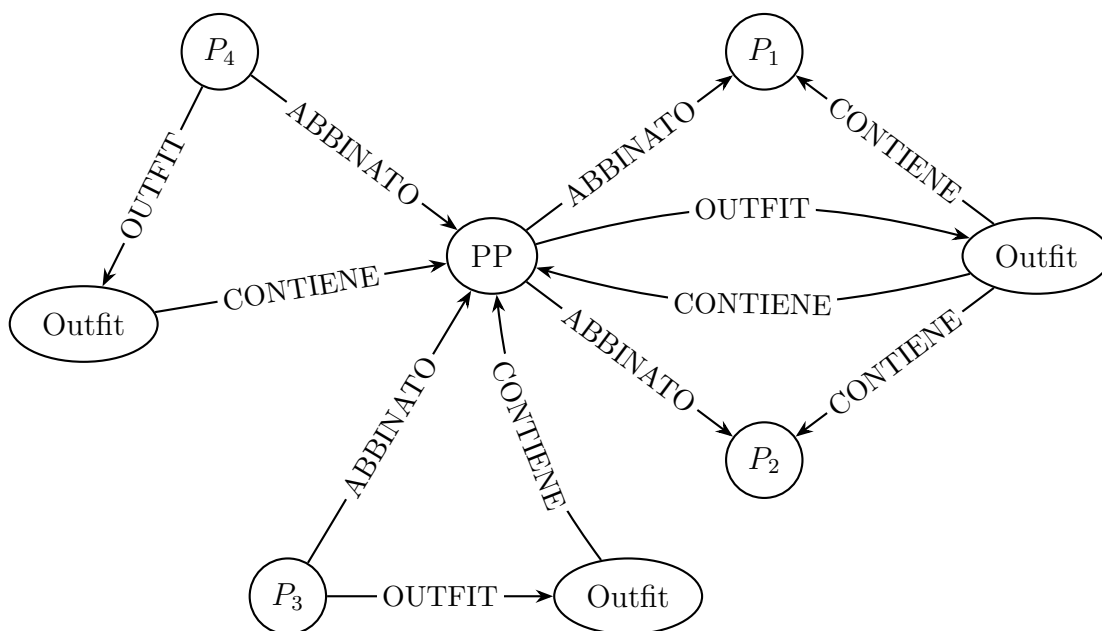


Figura 9.2

Ingrandendo sul grafo, possiamo notare come i nodi vengono messi in relazione tra di loro tramite una rappresentazione generica della struttura.



9.5.8 Limiti noti

Il lift non misura l'abbinamento. Quando ogni prodotto ha un solo outfit, n_{pc} vale 1 per quasi tutte le coppie e l'equazione si riduce a $N/(n_p n_c)$, cioè all'inverso della popolarità dei due capi. Ordinare i suggerimenti per lift favorirebbe i capi meno visti; la misura diventa informativa solo con più outfit per prodotto.

Due scale di somiglianza. La somiglianza a catalogo e quella di riferimento provengono da sistemi diversi e hanno soglie diverse, ma nel grafo occupano la stessa proprietà. Senza memorizzare la fonte, lo stesso valore può essere valido per una e insufficiente per l'altra.

Il massimo di sei strategie è sbilanciato. Le strategie che rimuovono il colore o la categoria partono avvantaggiate per costruzione, e il massimo alza la somiglianza di tutti i capi, rendendo necessaria una nuova taratura delle soglie. Una valutazione rigorosa richiederebbe un insieme di ritagli etichettato manualmente.

Occupazione e calzature. Una soglia di occupazione alta per calzature e accessori garantisce ritagli leggibili, ma ne scarta una parte consistente; molti outfit si riducono così a una coppia di capi invece che a un look completo.

Articoli non acquistabili. I capi introdotti dal ripiego sulle calzature possono appartenere ad altri marchi e vanno esclusi quando si propone un acquisto.

Bibliografia

- [1] Wikipedia. *Rete Neurale*. URL: https://it.wikipedia.org/wiki/Rete_neurale_artificiale.
- [2] Wikipedia Contributors. *Neural_Networks.svg*. URL: [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)#/media/File:Colored_neural_network.svg](https://en.wikipedia.org/wiki/Neural_network_(machine_learning)#/media/File:Colored_neural_network.svg).
- [3] IBM. *Modello di Reti neurali - IBM Documentation*. 2024. URL: <https://www.ibm.com/docs/it/spss-modeler/18.6.0?topic=networks-neural-model>.
- [4] IBM. *Model Training*. URL: <https://www.ibm.com/it-it/think/topics/model-training>.
- [5] Ultralytics. *Epoch in Machine Learning (ML)*. URL: <https://www.ultralytics.com/glossary/epoch>.
- [6] IBM. *Pytorch*. URL: <https://www.ibm.com/it-it/think/topics/pytorch>.
- [7] Ani Aggarwal. *YOLO*. URL: <https://medium.com/analytics-vidhya/yolo-explained-5b6f4564f31>.
- [8] Ultralytics. *Ultralytics YOLO*. URL: <https://docs.ultralytics.com/it/models/yolo26#ultralytics-yolo26>.
- [9] Wikipedia Contributors. *Precision&Recall.svg*. URL: https://en.wikipedia.org/wiki/Precision_and_recall.
- [10] Computer Vision Wiki. *Intersection over Union*. URL: <https://wiki.cloudfactory.com/docs/mp-wiki/metrics/iou-intersection-over-union>.
- [11] Computer Vision Wiki. *mean Average Precision*. URL: <https://wiki.cloudfactory.com/docs/mp-wiki/metrics/map-mean-average-precision>.
- [12] Miguel Morin e Matthew Willetts. *Non-Determinism in TensorFlow ResNets*. URL: <https://arxiv.org/abs/2001.11396>.
- [13] Thinking Machines Lab. *Defeating Nondeterminism in LLM Inference*. URL: <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>.

- [14] PyTorch. *Reproducibility — PyTorch Documentation*. URL: <https://docs.pytorch.org/docs/2.14/notes/randomness.html>.
- [15] Fabian Küppers. *Uncertainty Calibration and its Application to Object Detection*. URL: <https://arxiv.org/abs/2302.02622>.
- [16] Ultralytics. *SAM3*. URL: <https://docs.ultralytics.com/models/sam-3>.
- [17] Alexey Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. URL: <https://arxiv.org/abs/2010.11929>.
- [18] Enze Xie et al. *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*. URL: <https://arxiv.org/abs/2105.15203>.
- [19] Meta AI. *Segment Anything Model 3 (SAM 3): Segment Anything with Concepts*. URL: <https://ai.meta.com/research/>.
- [20] Alibaba Cloud. *QwenLM/Qwen3-VL - Official GitHub Repository*. URL: <https://github.com/QwenLM/Qwen3-VL>.
- [21] Ollama. *qwen3-vl Model Library*. URL: <https://ollama.com/library/qwen3-vl>.
- [22] LM Studio. *Qwen3-VL Features and Visual Agent Capabilities*. URL: <https://lmstudio.ai>.
- [23] Google AI for Developers. *Vision understanding | Gemma*. URL: <https://ai.google.dev/gemma>.
- [24] Joseph Redmon et al. *You only look once: Unified, real-time object detection*. URL: <https://arxiv.org/abs/1506.02640>.
- [25] Shilong Liu et al. *Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection*. URL: <https://arxiv.org/abs/2303.05499>.
- [26] yainage90. *fashion-object-detection*. URL: <https://huggingface.co/datasets/yainage90/fashion-object-detection>.
- [27] Qdrant. *Build a GraphRAG Agent with Neo4j and Qdrant*. URL: <https://qdrant.tech/documentation/examples/graphrag-qdrant-neo4j/>.